

PrimeTime Static Timing Analysis

A focused guide for ASIC physical design, STA and signoff engineers
Intermediate, Advanced and Expert level only

65

QUESTIONS

22

INTERMEDIATE

23

ADVANCED

20

EXPERT

50

DIAGRAMS

65 questions chosen from a bank of 400 for the mechanisms an interviewer actually probes. Every diagram is matched to its own question - 33 of the 50 were drawn specifically for this guide.



explore more at pdverse.net

How to use this guide

This is the short edition. Where the full bank covers 400 questions for reference and search, this guide keeps the 65 that teach a **mechanism** - the ones where an interviewer can keep asking "and why?" and find out whether you actually understand the tool.

Nothing here is beginner material. The mix is 22 intermediate, 23 advanced and 20 expert, and the order is deliberately shuffled across topics and levels: working straight through feels far more like a real interview than revising one subject at a time.

Each entry gives you a ten-second answer to lead with, the full explanation behind it, where the idea shows up in real physical-design and signoff work, the mistake that catches most candidates, and the follow-up question you should expect next.

On the diagrams. A shared diagram library gets stretched thin when one picture has to serve many questions. For this guide 33 diagrams were drawn from scratch so that every figure answers the question it sits under - the exception report line, the two NAND switching directions, the eight libraries a two-rail level shifter needs, and so on.

What is covered

5	Back Annotation — 8, 24, 31, 38, 64	6	Operating Conditions — 4, 39, 43, 52, 55, 59
4	Case & Mode Analysis — 29, 36, 44, 46	2	SMVA — 2, 28
8	Clocks — 1, 5, 17, 20, 32, 34, 37, 45	8	STA Fundamentals — 11, 14, 42, 49, 51, 58, 61, 63
5	Constraints — 3, 18, 35, 41, 56	7	Timing Exceptions — 13, 25, 27, 40, 48, 53, 62
7	Delay Calculation — 7, 12, 16, 22, 47, 57, 65	1	Tool Setup & Scripting — 54
1	Design Data — 60	6	Variation — 6, 9, 15, 26, 33, 50
5	Multivoltage — 10, 19, 21, 23, 30		



Questions

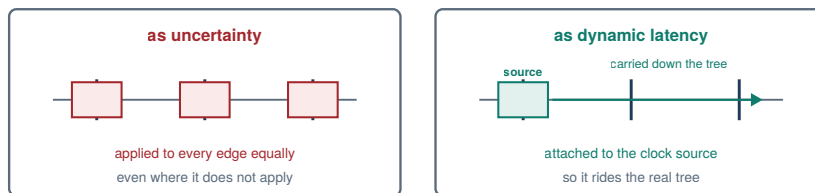
INTERMEDIATE

1 How would you model PLL clock jitter, and why use dynamic latency rather than uncertainty?

Model PLL jitter as a dynamic component of clock source latency with `set_clock_latency -dynamic`, because unlike `set_clock_uncertainty` it affects the crosstalk arrival-window calculation and is handled by CRPR.

The right home for PLL jitter is the dynamic part of the clock source latency, using the `-dynamic` option of `set_clock_latency`, rather than uncertainty alone. The reason is a subtlety worth knowing precisely. Clock uncertainty, set with `set_clock_uncertainty`, tightens the setup and hold checks, but it does not affect the calculation of crosstalk arrival windows and it is not considered by CRPR, clock reconvergence pessimism removal. Dynamic source latency behaves differently: `set_clock_latency -dynamic` is handled by CRPR in the same way as PrimeTime SI delta delays, so the jitter properly affects the crosstalk arrival-window calculation, and CRPR correctly handles both the static and the dynamic portions, removing the appropriate pessimism at the common clock point. For designs where jitter interacts with SI and crosstalk timing, that makes dynamic latency the more accurate model. In practice you write it alongside the static value: `set_clock_latency -source -early 2.5 -dynamic -0.5 [get_clocks CLK]` and `set_clock_latency -source -late 5.5 -dynamic 0.5 [get_clocks CLK]`, where the total early is static 3.0 plus dynamic -0.5 giving 2.5, and the total late is static 5.0 plus dynamic 0.5 giving 5.5. `report_timing` shows the static and dynamic components separately, and `report_crpr` reports the CRP values calculated for the static and dynamic conditions and which one was actually used.

Jitter as dynamic latency, not as uncertainty



Latency propagates through the network and interacts with CRPR; an uncertainty number does neither.

In PD / signoff. You make this choice when building the clock model for an SI-aware signoff run on a PLL-driven design. The jitter number comes from the PLL specification and goes in as `-dynamic` on the source latency rather than being folded wholesale into uncertainty; afterwards `report_crpr` is read to confirm the static and dynamic CRP values, and `report_timing` to see the two components broken out on real paths.

Trap. Rolling all the jitter into `set_clock_uncertainty` and assuming it is covered. It tightens the checks but stays invisible to CRPR and to the crosstalk arrival windows, so the SI analysis never sees it and the common-point pessimism removal is computed without it - a silent accuracy loss, not a warning.

Expect next. How does CRPR treat the dynamic latency component differently from the static portion at the common clock point?

Clocks / Generated Clocks · `set_clock_latency`, `get_clocks`, `report_crpr`, `set_clock_uncertainty`

ADVANCED

2 Explain the precedence interaction when an explicit `-sms_scenarios` option and an active `push_sms_scenario` context both apply to a command.

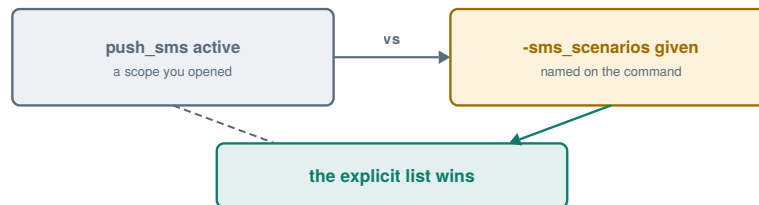
Explicit references have the highest precedence, so the command's own `-sms_scenarios` wins and the pushed context is ignored entirely for that command. `get_current_sms_scenario` still reports the pushed context, but it has no effect on the explicitly scoped command.

There are two ways a scenario reaches a command: explicitly, as `-sms_scenarios` on the command itself, or implicitly, from the surrounding context - including a context established by `push_sms_scenario`. The precedence rule is that explicit references sit at the top and implicit context references sit below them. When both apply, the explicit option on the command wins and the pushed context is completely ignored for that command. Not merged with it, not intersected - ignored. This is deliberate, and it follows the



same principle you see throughout constraint handling: the closest, most specific specification governs. A pushed context is a broad statement about a region of your script; an option typed on one command is a narrow statement about that command. When the two disagree, the narrower one is what the author most likely meant. One detail catches people during debug. `get_current_sms_scenario` continues to report the pushed context, because that query is describing the context stack, not the scoping decision any particular command made. So you can be inside a pushed context, query it, get the pushed scenario back, and still have the command on the next line operating under an entirely different scenario. That is not an inconsistency - the two are answering different questions. If you want to know what scoped a command, look at the command, not at the context query.

An explicit scenario list beats an active push



Naming scenarios on the command is the more specific statement, so it overrides the surrounding scope.

In PD / signoff. This matters when you wrap a shared constraint procedure in `push_sms_scenario` to retarget it. Any command inside that procedure carrying its own `-sms_scenarios` will ignore your push, so the retargeting silently only affects part of the procedure. Grep the procedure body for explicit `-sms_scenarios` before assuming a pushed context governs all of it.

Trap. Expecting a pushed context to override or combine with a command's explicit `-sms_scenarios`, or using `get_current_sms_scenario` as evidence of what scoped a command. The query reports the pushed context even when the command ignored it, which makes the debug trail look consistent while the scoping is not.

Expect next. Given that explicit options override a pushed context, how would you write a reusable constraint procedure whose scenario scoping the caller can actually control?

SMVA / SMVA Reporting · `get_current_sms_scenario`

INTERMEDIATE

3 How would you set up an ideal network and control its propagation?

Mark the source objects with `set_ideal_network`, and the ideal property propagates through the transitive fanout across combinational cells, stopping at sequential cells; use `-no_propagate` to confine it to the electrically connected nets and pins.

You specify sources, not regions. `set_ideal_network <source_ports/pins>` marks the listed objects ideal, and from there the property spreads through the transitive fanout, so it is worth knowing the propagation rules precisely. A pin is ideal if it is in the object list, or its driver cell is ideal, or it is a load on an ideal net. A net is ideal if all of its drivers are ideal. A combinational cell is ideal if all of its inputs are ideal. Propagation traverses combinational cells but stops at sequential cells, so the ideal property flows forward through combinational logic from the source and does not cross a register boundary - which is sensible, because the designation is about a combinational network driven from the source rather than about the registers. When you want less reach than that, use `-no_propagate`, which restricts the ideal property to just the nets and pins electrically connected to the source rather than the full transitive fanout; that is the right choice when only the immediate network should be ideal, for example one particular high-fanout net. So a typical setup is `set_ideal_network [get_ports P1]` to mark a port's whole downstream network, and `set_ideal_network -no_propagate [get_pins U_big/EN]` to mark only the enable net itself. You can then give the ideal network its own characteristics with `set_ideal_latency 0.5 [get_ports P1]` and `set_ideal_transition 0.1 [get_ports P1]` if you want an ideal latency or transition specified for it.

In PD / signoff. You meet this on pre-layout runs for high-fanout nets such as resets and enables, where you do not want an unbuilt network's delay dominating the report. The decision it drives is how much of the fanout cone leaves real timing analysis - so before signoff you audit which nets are still ideal, because an ideal network is exempt from real delay calculation.

Trap. Marking a source ideal and not realizing how far the property travelled. Without `-no_propagate` the whole downstream combinational cone up to the registers becomes ideal, so paths you expected to be analyzed silently use ideal timing and report no violation.

Expect next. Where does ideal-network propagation stop, and why does it stop there?



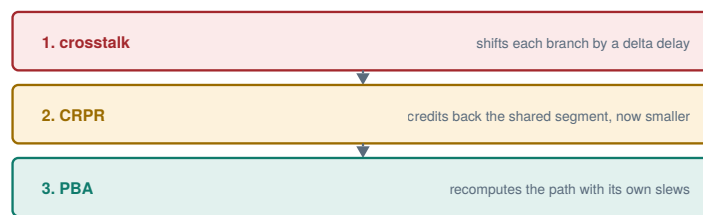
EXPERT

4 Explain how CRPR, crosstalk, and PBA interact for an SI-critical clock path, end to end.

Three layers: base CRPR removes the OCV double-count on the shared clock portion for final slack; CRPR plus crosstalk removes the coupling delta on the common segment only for zero-cycle checks; and in PBA, `pba_enable_xtalk_delay_ocv_pessimism_reduction` extends CRPR to the arrival windows.

There are three layers, and they stack. The base layer is ordinary CRPR: it removes the OCV double-count on the shared clock portion when final slack is computed, and it does so on every path whenever it is enabled. The second layer is CRPR interacting with crosstalk, governed by the zero-cycle rule. On the common segment, crosstalk-induced delay is removed only for zero-cycle checks, where the same edge launches and captures, because only then does the aggressor affect the launch and the capture signal identically. For any other check the two events are different edges at different times, the delta cannot be assumed identical, and it stays in. The third layer is arrival-window pessimism in path-based analysis. During PBA the arrival windows used for crosstalk timing do not get CRPR by default, so derating on a shared clock segment can falsely widen the aggressor and victim windows and manufacture overlaps that cannot physically occur. Setting `pba_enable_xtalk_delay_ocv_pessimism_reduction` to true applies CRPR to those windows as well. Getting an SI-limited clock path genuinely accurate can require understanding all three and, where appropriate, enabling all three - at escalating runtime cost, since each layer moves the CRPR accounting into a more expensive part of the computation.

Three refinements, applied in order, on one clock path



change the order and the numbers change

Each step consumes the output of the one above, so a slack comparison is only fair at the same stage.

In PD / signoff. This is how you triage an SI-limited clock path at post-route signoff. You establish that base CRPR is on, decide whether the check is zero-cycle to know if the coupling delta should be credited, and only then reach for arrival-window pessimism reduction in PBA. The layering tells you which runtime you are about to spend and what accuracy you get back.

Trap. Treating CRPR as one switch, so that enabling it is assumed to have removed all shared-clock pessimism including crosstalk deltas and arrival-window widening. Each layer is separately scoped, and the crosstalk layer is further restricted to zero-cycle checks, so a path can be CRPR-enabled and still carry two kinds of pessimism.

Expect next. For a non-zero-cycle SI-limited clock path, which of the three layers can still recover pessimism and which cannot?

Operating Conditions / On-Chip Variation · `pba_enable_xtalk_delay_ocv_pessimism_reduction`

INTERMEDIATE

5 How would you set interclock uncertainty between two clock domains, and when is it needed?

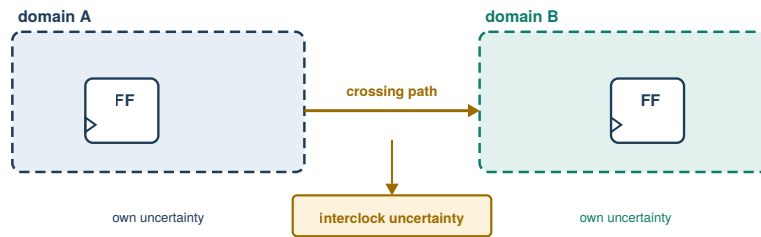
Use `set_clock_uncertainty` with `-from` and `-to`, as in `set_clock_uncertainty 2 -from [get_clocks CLKB] -to [get_clocks CLKA]`. It is needed for clock-domain-crossing paths, where launch and capture registers sit on different clocks.

For a path whose launch and capture registers are on different clocks, the relevant skew is the variation between two different clocks, and that is what `set_clock_uncertainty -from` and `-to` specifies. For example `set_clock_uncertainty 2 -from [get_clocks CLKB] -to [get_clocks CLKA]` sets the uncertainty for paths running from CLKB to CLKA. You need it because cross-domain variation can differ from the within-domain skew and is often larger: the two clock trees may share very little common path, and may derive from different sources, so their edges can move further relative to each other than edges within one tree do. Specify it per crossing that matters, one `-from` / `-to` pair at a time, since different crossings can carry different relative uncertainty. The relationship is directional - `-from` is the launch clock and `-to` is the capture clock - so if paths cross both ways you write both directions. A typical setup pairs a modest within-domain value with a larger interclock one: `set_clock_uncertainty 0.3 [get_clocks CLKA]`



and `set_clock_uncertainty 0.3 [get_clocks CLKB]` for the simple case, then `set_clock_uncertainty 0.6 -from [get_clocks CLKA] -to [get_clocks CLKB]` and the reverse. The precedence rule to remember is that when a path has both a simple and an interclock uncertainty, the interclock value takes precedence - so on a CLKA to CLKB path the 0.6 is used, not the 0.3.

Uncertainty between two clock domains, not inside one



Two clocks that are related but built by different trees need a larger allowance on the paths between them.

In PD / signoff. You write these when the clock architecture is settled and the list of timed clock crossings is known. Each synchronous crossing that is actually timed gets its own `-from / -to` pair, both directions where paths run both ways, and the value comes from how much common clock path the two trees really share. It is the difference between believable and fictional CDC slack.

Trap. Leaving cross-domain paths on the smaller within-domain value, which understates the real relative variation between two trees that share little common path and risks missed violations. The other half of the trap is forgetting the direction: a `-from A -to B` setting does nothing for paths running B to A.

Expect next. If a path has both a simple and an interclock uncertainty specified, which value does PrimeTime apply?

Clocks / Uncertainty & Jitter · `get_clocks`, `set_clock_uncertainty`

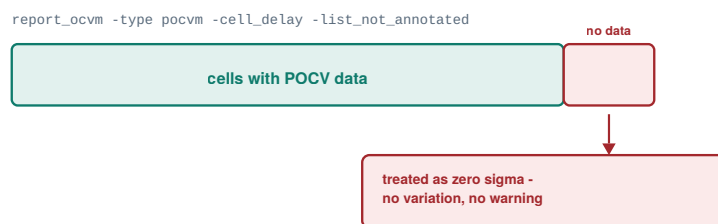
ADVANCED

6 Why is `report_ocvm -type pocvm -cell_delay -list_not_annotated` an important pre-signoff check, and what does a missing cell imply?

It lists cells with no POCV coefficient or distance data, which fall back to zero variation. Any path through such a cell has under-modeled spread, so its corner is optimistic - a silent hole that can hide a real violation behind a normal-looking report.

`report_ocvm -type pocvm -cell_delay -list_not_annotated` enumerates the cells that lack POCV coefficient or distance data - cells that neither LVF nor a side file annotated, and which therefore fall back to zero variation or to default behavior. A missing cell is a silent accuracy hole: it contributes no sigma to the statistical propagation, so every path through it has under-modeled variation and its corner value is optimistic, showing less spread than reality and potentially masking a real violation. What makes it dangerous is precisely that it fails quietly - the timing report looks entirely normal, just with distributions that are too tight, so nothing prompts you to look. The check does not need a timing update, so you can run it before `update_timing` and catch annotation gaps early, which is when they are cheap to fix. The usual causes are a library missing LVF tables for certain cells, a side-file `object_spec` that did not match the names you thought it would, or new cells introduced into the design after the POCV data was generated. The fix is to ensure every timing-relevant cell has POCV data, whether through complete LVF or through side-file coverage. Think of it as the POCV form of the general completeness discipline: the presence of variation data on every cell is what makes a statistical signoff trustworthy, and an un-annotated cell is a path carrying a hidden confidence gap that no slack number will ever reveal to you.

Cells with no variation data fail silently



A cell with zero sigma looks perfectly predictable, which is the most optimistic answer available.



In PD / signoff. Run it as the first gate of POCV signoff, before the first `update_timing`, and require a zero-length list or a written explanation per entry. When it flags cells, the triage is library coverage first, then side-file `object_spec` matching, then a check for cells added by recent ECOs. Archive the empty list as signoff evidence.

Trap. Skipping the check because the timing reports look fine. They will look fine - an un-annotated cell removes sigma rather than adding an error message, so the slack is optimistically tight and nothing in a normal report distinguishes it from a genuinely clean path.

Expect next. If the not-annotated list names only a handful of non-critical cells, how would you decide whether to waive them?

Variation / AOCV Tables · `report_ocvm`, `update_timing`

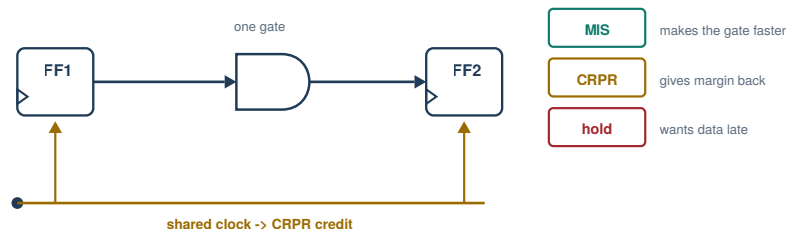
EXPERT

7 Explain how MIS, CRPR, and the hold check interact on a short clock-adjacent data path.

All three land on min delay. MIS speeds up co-switching data gates so data arrives earlier, which tightens hold; CRPR removes shared-clock-segment pessimism from the launch and capture timing; the hold check then compares the MIS-shortened arrival against the CRPR-adjusted capture.

Three effects converge on the same quantity - min delay, which is what the hold check consumes. MIS reduces data-path gate delays wherever multiple inputs co-switch, the Case-B speed-up, and the consequence is that data arrives earlier. Earlier data is the dangerous direction for hold, so MIS tightens the check. CRPR works on the clock side: it removes the pessimism of treating the shared segment of the launch and capture clock paths as independent, adjusting the launch and capture timing on the common portion, and for a zero-cycle hold check it also reconciles crosstalk on that common segment. The hold check then compares the MIS-shortened min data arrival against the CRPR-adjusted capture time. The subtlety worth voicing is that the two effects do not push the same way. MIS makes data faster and so tightens hold unambiguously; CRPR removes launch and capture pessimism, which can loosen or tighten hold depending on the shared segment. Both therefore have to be modelled to get the true hold margin. Leave MIS out and you are optimistic, because data really can arrive earlier than you modelled. Mishandle CRPR and you carry false pessimism instead. On a short clock-adjacent path where a few picoseconds decide the verdict, getting both right is precisely what separates a real hold violation from a phantom - which is why MIS is scoped to hold in the first place and why CRPR has a specific zero-cycle crosstalk rule.

Three effects landing on one short hold path



On a one-gate path there is almost no delay to absorb them, so all three show up in the slack at once.

In PD / signoff. This is a post-route hold-signoff debug situation. On a short path that fails hold by picoseconds, you check that MIS is active on the data gates and that CRPR is enabled with its zero-cycle crosstalk handling, then read the launch and capture clock legs to see how much common-path credit was applied. The decision it drives is whether to spend a hold buffer or dismiss the violation as pessimism.

Trap. Disabling MIS to make a short hold path pass, or crediting CRPR without checking the zero-cycle crosstalk treatment on the common segment. Turning MIS off is optimistic - real data can arrive earlier than the model - while a mishandled CRPR leaves false pessimism that gets paid for in unnecessary hold buffers.

Expect next. On a zero-cycle hold check, why does crosstalk on the common clock segment need reconciling rather than simply cancelling out?

Delay Calculation / Interconnect & Ceff

ADVANCED

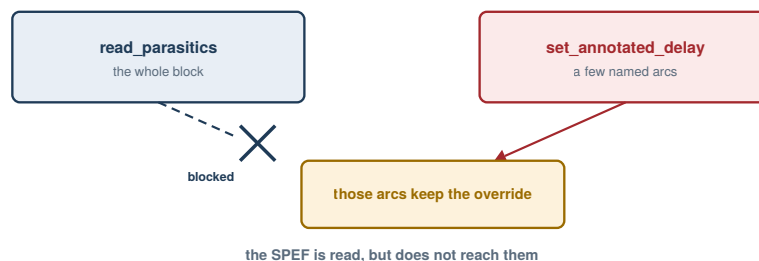


8 A design has both `set_annotated_delay` on some arcs and a full SPEF read. How does PrimeTime resolve the overlap, and what's the debugging implication?

Annotated delays win. `set_annotated_delay` is tier 1 and `read_parasitics` detailed parasitics are tier 3, so on any arc where both apply the annotated delay is used and the SPEF-derived delay for that arc is ignored.

Precedence decides this, not read order. `set_annotated_delay` sits at tier 1 and detailed parasitics from `read_parasitics` sit at tier 3, so wherever the two overlap, PrimeTime uses the annotated delay and ignores the SPEF-derived delay on that arc. The SPEF is not discarded - it still supplies R and C for design-rule checking and it still drives every other arc - but on the annotated arc its contribution to delay is simply not consulted. The debugging implication is where the real value of this question lies. A stray `set_annotated_delay`, say a debug override someone left behind from an earlier session, freezes that arc's delay at the annotated value no matter what parasitics you subsequently read. The symptom presents as my SPEF delay is not showing up on this specific arc, and it is maddening precisely because everything about the parasitic read looks correct. The move is to check `report_annotated_delay` for a manual annotation shadowing the arc. The fix is `remove_annotated_delay` on that arc, or a broader reset with `reset_design` or `reset_design -keep_parasitics` if you want to clear the session while holding on to what you read. It is the same shadowing hazard as `set_load` beating SPEF capacitance, one tier further up.

An explicit arc override survives a full SPEF read



In PD / signoff. This is a debug reflex for a single wrong arc in a session that otherwise annotates fine. Before re-extracting or re-reading anything, run `report_annotated_delay` and look for a manual annotation on the arc in question, then clear it with `remove_annotated_delay`. It also argues for keeping debug overrides out of shared constraint and annotation scripts, since they outrank everything the parasitic flow produces.

Trap. Assuming a later `read_parasitics` overwrites an earlier `set_annotated_delay` because it came second. Precedence, not order, decides: the tier-1 annotation stays in force, the arc's delay is frozen, and the parasitic read appears to have had no effect on that one arc.

Expect next. Why does a completeness check on the parasitics still pass on an arc whose delay is being shadowed by `set_annotated_delay`?

Back Annotation / SPEF & Parasitics · `set_annotated_delay`, `reset_design`, `read_parasitics`, `report_annotated_delay`

INTERMEDIATE

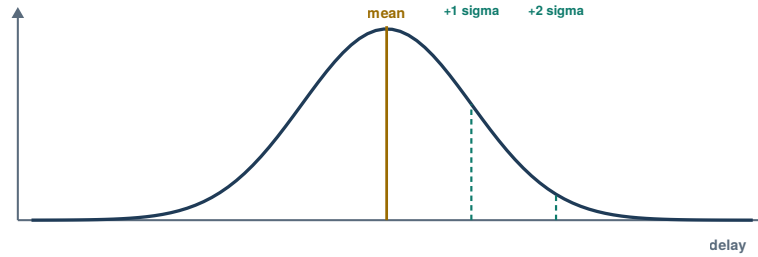
9 Why must you query `.mean/.std_dev` on POCV attributes instead of the bare attribute?

Because under POCV a slack or arrival attribute is a distribution rather than a scalar, so there is no single value to return - the bare attribute yields an empty list `{}`. `.mean` returns the average and `.std_dev` returns the sigma.

In a deterministic analysis an arrival or slack attribute is one number, so querying it directly makes sense. Under POCV it is not a number at all - it is a statistical distribution, and the tool has nothing sensible to hand back as a scalar. Querying the bare attribute therefore returns an empty list, shown as `{}`, which people frequently misread as "the attribute does not exist" or "POCV is not enabled". It does exist; you simply have to ask for a specific statistic of it. The `.mean` sub-attribute returns the average of the distribution and `.std_dev` returns the sigma. This convention applies across the POCV attributes: the `{min,max}_{rise,fall}_variation_{arrival,slack}` family on pins, and `variation_arrival` and `variation_slack` on timing paths and timing points. So the pattern in a script is always to append the statistic you want rather than to read the attribute raw. Practically this also tells you something about how to write POCV post-processing: because you get mean and sigma separately, any sorting, thresholding or margin arithmetic you do in Tcl has to be explicit about which statistic it is using, and a script written for deterministic slack will silently produce empty results rather than wrong ones when pointed at a POCV run.



A cell delay as a distribution, not a number



POCV carries a mean and a sigma for each arc and propagates the distribution, so depth averaging comes out of the arithmetic itself.

In PD / signoff. You hit this the first time you write a Tcl script to collect POCV slacks for a block summary. The loop returns nothing until you query `variation_slack.mean` and `variation_slack.std_dev`, and then you can rank endpoints by mean slack or by sigma to see which failures are variation-driven rather than nominal.

Trap. Reading the empty list {} from the bare attribute as evidence that POCV is off or the attribute is unsupported, and abandoning the query. The distribution is there - only `.mean` and `.std_dev` can be returned as scalars, and a script that forgets the suffix fails silently with empty output rather than an error.

Expect next. Which POCV attributes does this mean and std_dev convention apply to on pins as opposed to timing paths?

Variation / POCV Distributions · `variation_arrival`, `variation_slack`

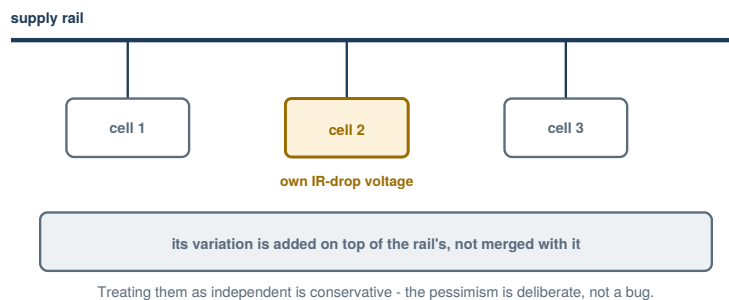
ADVANCED

10 Why is a per-cell IR-drop annotation treated as uncorrelated with the main rail, and what's the consequence?

Because a local IR drop is a physically independent droop - one instance's grid sagging says nothing about another's - so PrimeTime compares the full worst-case difference across the annotation instead of assuming driver and load track together.

Correlation is a physical claim, not a convenience. When two cells share a UPF supply net, their voltages genuinely move together and PrimeTime can safely ignore the difference. A per-cell IR-drop annotation is the opposite situation: it describes a local droop on one instance's grid, and there is no physical reason U3's voltage sagging implies that U1's does. The two values are independent, so for signal-level checking the tool deliberately compares the full worst-case difference across the annotation. Take a crossing where the annotation produces a driver at 0.8 against a load at 1.1 in one direction and 1.2 against 0.7 in the other: both comparisons are taken at their worst, and both are reported as violations. The alternative - assuming driver and load track each other - would be optimistic in exactly the case the annotation was created to model, and would hide a real electrical mismatch between two cells that can, in fact, be at different voltages at the same instant. The consequence for you is that annotating IR drop can legitimately introduce signal-level violations on crossings that were clean before, and those violations are not artifacts to be filtered away. Contrast the two rules once more: a shared UPF net is correlated and stays quiet, a per-cell annotation is uncorrelated and speaks up.

A per-cell voltage is treated as independent of the rail



In PD / signoff. This shapes how you read signal-level results after annotating IR drop with `set_voltage -cell -pg_pin_name`. New violations on previously clean domain crossings are the expected behaviour, and each one is a real worst-case pairing to assess - either the grid needs work at that instance or the crossing needs a level shifter, not a threshold adjustment to make the report quiet.

Trap. Treating violations that appear only after IR-drop annotation as pessimism and widening thresholds to suppress them. The worst-case comparison is intentional, because the two cells' droops are independent; suppressing it hides a mismatch that can genuinely occur.

Expect next. Why does the same nominal voltage difference stay quiet when both cells share one UPF supply net?

Multivoltage / UPF & Power Domains · `set_voltage`

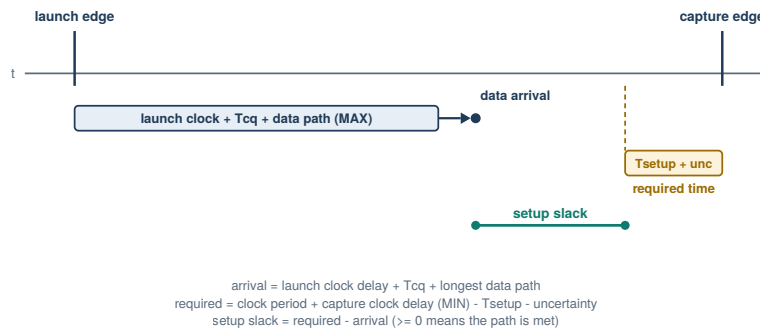
INTERMEDIATE

11 Explain the complete setup check for a register-to-register path, including which path delays are used.

Setup compares the latest data arrival at FF2/D against the earliest required time: $\text{arrival} = \text{launch clock delay} + T_{cq} + \text{max data path}$, $\text{required} = \text{period} + \text{min capture clock delay} - T_{\text{setup}} - \text{uncertainty}$, and $\text{slack} = \text{required} - \text{arrival}$.

A setup check on a register-to-register path (FF1 -> logic -> FF2) asks whether data launched by FF1 lands at FF2's `D` pin early enough before the capturing clock edge. The tool builds two numbers and subtracts them. The data arrival time at FF2/D is the clock arrival at FF1's clock pin (the launch clock path delay) plus FF1's clk-to-Q delay plus the combinational delay from FF1/Q to FF2/D. Because setup is a maximum-delay check, the tool takes the longest data path and the clk-to-Q, deliberately making the data as late as possible. The data required time at FF2/D is the capture edge time - one clock period later by default - plus the capture clock path delay, minus FF2's setup time and minus any uncertainty; here the tool uses the shortest capture clock path so the deadline arrives as early as possible. Written out: $\text{arrival} = \text{clk_launch} + T_{cq} + \text{data_path}(\text{MAX})$, $\text{required} = \text{period} + \text{clk_capture}(\text{MIN}) - T_{\text{setup}} - \text{uncertainty}$, $\text{setup slack} = \text{required} - \text{arrival}$, and $\text{slack} \geq 0$ means met. With $\text{clk_period}=10$, $\text{clk_launch}=0.5$, $T_{cq}=0.3$, $\text{data_path}=8.0$, $\text{clk_capture}=0.4$, $T_{\text{setup}}=0.5$ and $\text{uncertainty}=0.1$, $\text{arrival} = 0.5 + 0.3 + 8.0 = 8.8$, $\text{required} = 10 + 0.4 - 0.5 - 0.1 = 9.8$, so $\text{setup_slack} = 9.8 - 8.8 = +1.0$ and the path is met. The reason for that lopsided pairing is that if the path survives the slowest data against the earliest deadline, it survives every other combination.

Setup check: latest data arrival vs earliest required time



In PD / signoff. This equation is exactly what a `report_timing -delay_type max` path breakdown prints, line by line: the launch clock delay and clk-to-Q at the top of the arrival section, the accumulated cell and net delays, then the capture edge, capture clock delay, library setup time and uncertainty in the required section. Reading it lets you decide whether a failing path needs data-path optimization, clock rebalancing or a constraint fix.

Trap. Mixing the worst cases - pairing the longest data path with the longest capture clock path, which optimistically inflates the required time and hides a real setup violation. The other frequent slip is forgetting that setup time and uncertainty are subtracted from the required time, not added to the arrival.

Expect next. Why does setup use the longest data path but the shortest capture clock path?

STA Fundamentals / Setup Checks · `report_timing`, `delay_type`

ADVANCED

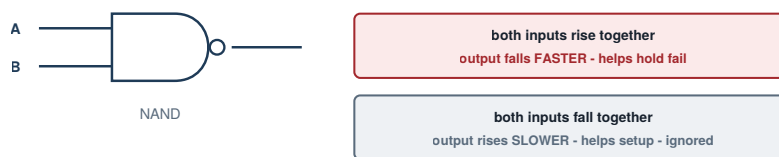


12 Derive why MIS is scoped to hold and not setup, using the two NAND switching directions.

For a low-to-high output the parallel pull-ups make multi-input switching faster, and a hold check uses min delay, so ignoring it would be optimistic. The high-to-low direction slows the cell, which affects setup, but setup paths are long enough that one gate is negligible.

Work the two switching directions of a NAND separately and the scoping falls out. When the output goes low to high, the pull-ups act in parallel, so all inputs switching together gives a shorter delay than one input switching. A hold check uses min delay. If the tool used the single-input, longer delay while the hardware can actually make the faster multi-input transition, the hold check would be optimistic: it would believe the data arrives later than it really can, and a genuine hold failure would be masked. That is the dangerous kind of error, so multi-input switching has to correct it. When the output goes high to low, multi-input switching gives a longer delay, which is a max-delay effect and would therefore touch setup. But setup-critical paths are long, with many gates in series, so one gate's small delay increase is a negligible fraction of the total and does not repay the runtime it would cost. The asymmetry is reinforced by path length on the other side: hold-critical paths are short, only a few gates, so one gate's delay decrease is a large fraction of the path and can flip the result outright. The entire payoff sits on the hold and min-delay side, which is exactly where MIS is scoped.

Why multi-input switching is a hold-side effect



Only the faster direction is dangerous, and the faster direction is exactly what a hold check fears.

In PD / signoff. Practically this means you enable MIS as part of hold signoff and do not expect it to move setup slack. When hold slack shifts after turning MIS on, that is the feature working as intended on short paths; if you see setup numbers move, question the setup rather than the physics.

Trap. Expecting MIS to relieve setup because multi-input switching also slows the high-to-low transition. It does, but on long setup paths the effect is a negligible fraction of the total, and the direction that matters is the faster low-to-high transition, where omitting MIS makes hold optimistic and can hide a real failure.

Expect next. Why is a short hold-critical path so much more sensitive to one gate's delay change than a long setup path?

Delay Calculation / General

INTERMEDIATE

13 When you pass a sequential cell to `set_max_delay -from / -to`, what happens under the hood?

The cell is expanded to pins: `-from cell` becomes each clock input pin of the cell, and `-to cell` becomes each data input pin. You can see the expanded pin-by-pin exceptions with `report_exceptions` or `write_sdc`.

Passing a sequential cell to `set_max_delay` is convenient shorthand, but PrimeTime does not keep the exception at the cell level - it expands it into pins. On the `-from` side, the cell expands to each clock input pin of that cell, since that is where a path launched by a sequential element actually starts. On the `-to` side, the cell expands to each data input pin, since that is where a path into a sequential element ends. So one command against a multi-bit or multi-port cell can become several pin-level exceptions, and you can see exactly what you got by running `report_exceptions` or `write_sdc` and reading the expanded form back. That is worth doing rather than assuming, because the expansion is where cell-level shorthand and pin-level reality can diverge. It also matters across tools. Some tools, including IC Compiler II, keep the exception at the cell level instead of expanding it to pins, which means PrimeTime and ICC2 can resolve overlapping exceptions differently - the same SDC, the same design, different winners when two exceptions contend on the same path. When the two tools disagree on a borderline path, that difference in exception representation is a prime suspect.



A point-to-point constraint on a flop replaces the clock requirement



`set_max_delay 12 -from FF4 -to FF5`

the clock-derived requirement is replaced, not added to -
and the hold check becomes a min-delay check of 0 unless you set one

In PD / signoff. You use this when reconciling a path that PrimeTime and IC Compiler II report differently. Dump the constraints with `write_sdc` or inspect `report_exceptions` in PrimeTime to see the pin-level expansion, then compare against how the implementation tool held the same exception - overlapping exceptions on the same path are where the two resolve differently.

Trap. Assuming cell-level exception shorthand means the same thing in every tool. PrimeTime expands to clock pins and data pins while some tools keep it at the cell, so overlapping exceptions can resolve to different winners and a borderline path looks clean in one tool and violating in the other.

Expect next. Why does the cell-versus-pin representation matter when PrimeTime and IC Compiler II disagree on a borderline path?

Timing Exceptions / Max/Min Delay · `set_max_delay`, `report_exceptions`, `write_sdc`

ADVANCED

14 How does PrimeTime model and analyze clock networks, including latency and uncertainty?

PrimeTime propagates a defined clock waveform through the network to get each register's latency - source plus network - derives skew from latency differences, and adds clock uncertainty for jitter and margin on top, using ideal latency pre-CTS and propagated clocks post-CTS.

PrimeTime models a clock as a defined waveform, a period with edges, at a source, then propagates it through the clock network to every register. The clock's arrival at a register is its latency, and latency has two parts: source latency, from the ideal clock origin to the clock definition point - an off-chip or PLL-to-port delay, for instance - and network latency, from the definition point through the on-chip clock tree of buffers and inverters to the register clock pin. Pre-CTS the network latency is ideal or estimated and you specify it with `set_clock_latency`; post-CTS it is propagated, computed from the actual clock-tree cells and parasitics, via `set_propagated_clock`. Differences in latency between registers are what produce skew. On top of that deterministic latency, PrimeTime adds clock uncertainty with `set_clock_uncertainty` - a margin covering jitter, the cycle-to-cycle edge variation from the clock source, an estimated-skew placeholder pre-CTS, and any added safety margin. Uncertainty is subtracted from the setup budget and added for hold, so it tightens both checks. Alongside this the tool handles CRPR, which removes common-clock OCV pessimism, and clock reconvergence generally. The pre-CTS to post-CTS move is therefore not a switch of one setting but the transition from estimated clock timing to signoff-accurate clock timing, with uncertainty shrinking as real skew becomes visible.

What one uncertainty number absorbs



Uncertainty is subtracted from the required time on a setup check and added on a hold check, so one number tightens both directions without touching any path delay.



In PD / signoff. Concretely this governs which clock settings belong to which stage: `set_clock_latency` and a generous uncertainty pre-CTS, then `set_propagated_clock` with a reduced, justified uncertainty once the tree exists. Getting the handover wrong in either direction is a common source of slack that moves for no design reason - and of hold problems appearing only after CTS.

Trap. Leaving the pre-CTS skew allowance inside `set_clock_uncertainty` after switching to propagated clocks. The real skew is then counted twice - once through propagated latency differences, once through the placeholder margin - producing pessimism that no amount of datapath work can close.

Expect next. What is the difference between an ideal and a propagated clock, and when does each belong in the flow?

STA Fundamentals / Slack & Arrival · `set_clock_latency`, `set_propagated_clock`, `set_clock_uncertainty`

INTERMEDIATE

15 Reconstruct the AOCV incremental derating: cell u1/u252 has late derate 1.082, early 0.924; you apply -increment -late 0.03 and -increment -early -0.03.

Increments add to the existing factors, so the final late derate is $1.082 + 0.03 = 1.112$ and the final early derate is $0.924 + (-0.03) = 0.894$. A result below 0.0 would be clamped to 0.0.

The key word is `-increment`. It does not replace the derate already in effect on the cell, and it does not multiply it - it is added arithmetically to it. So for `u1/u252`, which already carries a late derate of 1.082 and an early derate of 0.924, applying `-increment -late 0.03` gives a final late factor of $1.082 + 0.03 = 1.112$, and applying `-increment -early -0.03` gives a final early factor of $0.924 + (-0.03) = 0.894$. Note that the early increment is negative, which is what makes the early factor smaller and therefore the early path faster - the two increments push the window open in both directions by the same 0.03. There is one guard on the arithmetic: if the sum came out below 0.0, the tool clamps the resulting factor to 0.0 rather than letting a negative derate produce a negative delay. Understanding this as addition rather than substitution matters because incremental derating is how you layer a small extra guardband on top of a table-driven or library-driven derate without regenerating the underlying data - you keep the characterized factor and nudge it, and the reported factor is the sum, not the increment you typed.

In PD / signoff. You meet this when signoff asks for a small extra guardband on a specific block or cell class without touching the variation tables. You apply the increment, then confirm with `report_ocvm` or the derate section of a delay-calculation report that the factor shown is the sum you expected, not the raw increment. Getting the arithmetic right is what lets you defend how much margin you actually added.

Trap. Assuming `-increment` overrides the existing derate, so people expect to see 0.03 and 0.97 in the report instead of 1.112 and 0.894. The related error is giving the early increment as a positive number, which tightens the early side rather than loosening it and quietly removes margin instead of adding it.

Expect next. If a chain of increments drove the early factor below zero, what value would the tool actually use in delay calculation?

Variation / AOCV Basics · `report_ocvm`

EXPERT

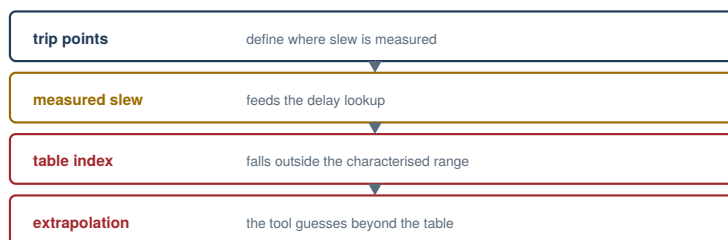
16 Walk through how trip points, slew, and the RC-011 extrapolation limit chain together into a single accuracy failure mode.

Wrong trip points corrupt the measured slew, that slew indexes the driver or CCS tables out of range, delay calc extrapolates (RC-011) and emits a larger output slew, which pushes the next stage further out of range - the error compounds down the path.

It is a chain, and each link amplifies the previous one. Trip points define how slew is measured, so if a slew is taken across an unexpected threshold pair the slew value handed to delay calculation is already wrong. That slew is then the index into the driver or CCS tables. If it lands beyond the maximum characterized slew index, delay calculation has no data and must extrapolate, which is what `RC-011` reports: a less accurate delay, and - crucially - a larger output slew. That larger slew becomes the input slew for the next stage, indexing its tables even further out of range, producing another `RC-011` and another inflated slew. So a subtle trip-point issue cascades into a chain of extrapolations, each stage worse than the last. Breaking the chain has three steps. First, confirm the trip points, meaning the `-thresholds` settings, are correct so slews are measured across the intended pair. Second, set `max_transition` from the lowest maximum slew index in the libraries so propagated slews stay inside the characterized range. Third, treat `RC-011` as a real signal and fix the library index range rather than widening the clip to silence it. The reason this failure mode is dangerous is that it is invisible if you inspect only one stage; the damage is done by the propagation of an out-of-range slew, not by any single delay number looking obviously wrong.



One wrong trip point becomes a whole-design error



Every stage inherits the error, which is why the whole design looks slightly off rather than one path.

In PD / signoff. In post-route signoff you see this as a cluster of RC-011 messages down one path. The move is to check `-thresholds` against the library's characterization thresholds, compare `max_transition` against the lowest maximum slew index across the linked libraries, and use `report_delay_calculation` on the first offending stage rather than the worst-slack stage - the first out-of-range index is where the chain starts and where the fix belongs.

Trap. Chasing RC-011 at the stage with the worst slack, or widening the slew clip to make the messages go away. That hides the compounding rather than stopping it - the real defect is usually a trip-point mismatch or a `max_transition` set above the library's lowest maximum slew index, several stages upstream.

Expect next. Given a path with RC-011 at three consecutive stages, how would you determine which stage first went out of range?

Delay Calculation / CCS Modelling · `report_delay_calculation`

INTERMEDIATE

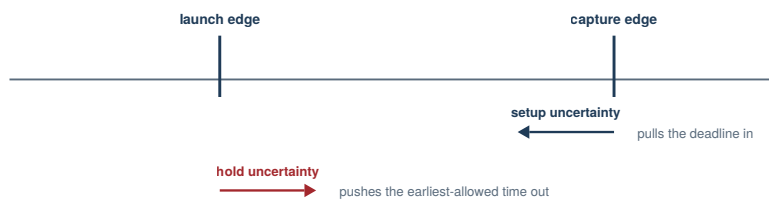
17 How would you constrain a design with both setup and hold clock uncertainty and explain the values?

Give separate values, for example `set_clock_uncertainty -setup 0.25 [get_clocks CLK]` and `set_clock_uncertainty -hold 0.10 [get_clocks CLK]`. Setup uncertainty is subtracted from the required time, hold uncertainty is added, and each covers its own relevant skew plus margin.

Specify `-setup` and `-hold` separately, with values chosen to model the real skew plus the right margin for each check. Setup uncertainty covers the clock skew plus any setup margin or guardband, and it is subtracted from the required time, which tightens the setup or max-path check; it typically bundles the expected skew, jitter, and a margin for modelling and OCV. Hold uncertainty covers the skew relevant to hold plus a hold margin, and it is added to the required time, tightening the hold or min-path check. Hold is often given a smaller number than setup, because the skew components that affect hold differ and hold margins, while tight, are usually smaller in absolute terms. The two checks are worst-cased in opposite directions, which is the underlying reason they deserve independent values: setup is worst-cased by the capture clock arriving early relative to launch, hold by the capture arriving late, so the components that matter and their worst-case directions are not the same. Forcing one number to serve both would over-constrain one check and under-constrain the other. Timing in the flow matters too. Pre-CTS these are estimates covering the full anticipated tree skew, so they are relatively large. Post-CTS, once the clocks are propagated, the real skew is captured directly by the computed per-register latencies, so both values drop to residuals covering only jitter, OCV variation and modelling guardband - keeping the pre-CTS numbers would double-count the skew.



One command, two numbers, opposite directions



```
set_clock_uncertainty -setup 0.12 -hold 0.05
```

Both tighten the design; giving only one leaves the other check unmargin.

In PD / signoff. These two lines sit in the clock section of every SDC and directly move both slack numbers, so they are reviewed at each milestone. Pre-CTS the values come from the target skew budget; post-CTS you cut them to residuals when you add `set_propagated_clock`. An inherited SDC with a large hold uncertainty and propagated clocks is a standard cause of unclosable hold.

Trap. Applying a single value to both checks, or carrying the pre-CTS numbers into propagated post-CTS runs. The second double-counts skew - once in the propagated latency differences and again in the uncertainty - making both checks needlessly pessimistic while looking like a legitimately hard design.

Expect next. Why is the worst-case direction of clock skew opposite for setup and hold, and how does that shape the two numbers you choose?

Clocks / Uncertainty & Jitter · `get_clocks`, `set_clock_uncertainty`, `set_propagated_clock`

EXPERT

18 How should constraint rigor differ for a safety-critical versus a consumer design?

The mechanics are identical; verification exhaustiveness, budget justification, exception rigor, traceability and independent review scale with consequence. Safety-critical demands justified context-validated budgets, zero-tolerance completeness, individually reviewed exceptions and auditable tapeout-consistent constraints; consumer applies the same methodology proportionately.

Nothing about the constraint commands changes between a brake controller and a phone SoC. What changes is how much evidence you must produce that the constraints are right. For safety-critical work the rigor is maximal. Boundary budgets are accurate and context-validated - no arbitrary fractions - with each budget justified against the real context and documented. Verification covers both completeness, with zero tolerance for unconstrained paths, and correctness, with budgets validated, exceptions reviewed and justified, and consistency proven; these are mandatory, evidenced gates, not best-effort checks. Exception discipline is rigorous: every `set_false_path` and `set_multicycle_path` is individually justified, independently reviewed for correct scope, and documented, because exceptions suppress checking legitimately and therefore stay invisible to completeness checks - an over-broad exception is a silent hole in the verification, and here that hole is a potential undetected hazard. Constraints are traceable: which constraint set and version was used for each qualified signoff run, recorded and auditable, with proof that the signoff constraints are consistent with the taped-out design. The constraints get independent review as rigorously as the RTL, because they are part of the safety case. For a consumer design you run the same methodology with proportionate rigor - accurate budgets, verification gates, reasonable exception discipline - but lighter formal justification, traceability and independent review, optimizing cost and schedule, because a constraint error there is a quality and rework risk rather than a safety one. One discipline, scaled to consequence.

In PD / signoff. On a safety-critical program every signoff run archives its exact constraint version alongside the reports, and each exception carries a justification record with a reviewer name. The practical decision this drives is whether a run can be cited as qualification evidence: an unreviewed exception or an untraceable constraint version disqualifies the run even if the timing is clean.

Trap. Assuming safety-critical rigor means tighter margins. The distinctive additions are justified and validated budgets, exhaustive correctness verification, individually reviewed exceptions, traceability and independent review - adding pessimism while leaving an unreviewed wildcard `set_false_path` in place leaves the actual hazard, an unchecked real path, fully intact.

Expect next. What constraint traceability does a certification audit actually require from each qualified signoff run?

Constraints / General · `set_false_path`, `set_multicycle_path`, `check_timing`

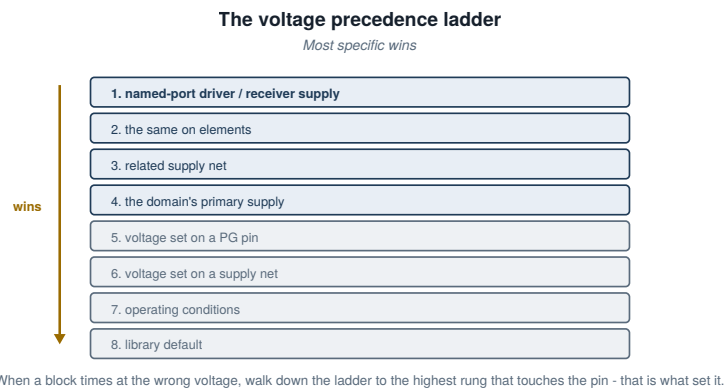
INTERMEDIATE



19 Explain the voltage precedence ladder. Why does it exist?

It is the deterministic tie-breaker among the many mechanisms that can set a voltage on the same object, ordered so that the more specific and explicitly named a setting is, the higher it wins - a targeted override always beats a broad default.

Several different mechanisms can each claim to set the voltage on one object, so the tool needs a fixed order to decide which one actually applies. From highest to lowest the rungs are: `-driver` or `-receiver_supply` on named ports; the same options on elements; `set_related_supply_net`; the domain's primary supply; `set_voltage` on a PG pin, which needs an SI license; `set_voltage` on a supply net; `set_operating_conditions`; and finally the library `voltage_map` default. Read down that list and the organising principle is clear: the more specific the target and the more explicitly you named it, the higher it sits. A voltage you set on a named port outranks one you set on an element, which outranks a related-supply-net assignment, which outranks the domain's primary supply, and everything above outranks the broad `set_operating_conditions` and the library default at the bottom. The ladder exists so that behaviour is predictable rather than order-of-command dependent - a targeted override you wrote for one port always beats the broad default that covers everything, and it does so deterministically every run. That predictability is what makes the ladder a debug tool as well as a rule: when a path times at the wrong voltage, you walk down the ladder to the highest rung that touches the pin in question, and that is what is setting it.



In PD / signoff. You use this in multivoltage signoff when a path's delays look wrong for the domain it sits in. Rather than guessing, you walk the ladder against that pin - checking for a `-driver` or `-receiver_supply` on the port, then `set_related_supply_net`, then the domain primary supply, then `set_voltage` and `set_operating_conditions` - and the highest rung that touches the pin is the answer. That decides whether the fix is a constraint change or a UPF issue.

Trap. Assuming the last voltage command you wrote wins, or that a broad `set_operating_conditions` overrides earlier per-port settings. It does not - it sits near the bottom of the ladder, above only the library `voltage_map` default, so a `-driver` or `-receiver_supply` on a named port silently outranks it no matter what order the commands were issued in.

Expect next. A path times at the wrong voltage - how would you walk the ladder to find what is actually setting it?

Multivoltage / Supply Network · `set_voltage`, `set_related_supply_net`, `set_operating_conditions`

EXPERT

20 How would you defend investment in clock methodology, tooling, and verification to an executive?

Frame it as insuring the foundation of the whole signoff: every check is measured against clocks, clock errors are systemic and silent, and the methodology cost is modest and amortized against respin-scale risk.

The executive case rests on five points. First, clocks are the reference for all timing: every setup and hold check depends on the clock model, so a clock error - a missing generated clock, ideal clocks left post-CTS, a wrong clock group - systemically corrupts many paths at once. Correct clocks are a precondition for the entire signoff being valid, not a refinement of it. Second, the combination of silent and systemic is what makes the risk large. Clock errors produce no obvious error message and they affect whole domains and crossings, so without rigorous methodology and verification a clock error propagates undetected all the way to tapeout - either as a missed violation, especially skew-driven hold that is frequently unfixable post-silicon, or as floods of false violations that consume closure. Third, it saves schedule rather than costing it: correct clock groups eliminate spurious cross-domain violation floods that otherwise bury the real issues and waste weeks, and accurate clocks point closure at real problems. Fourth, the cost is cheap and amortized - the clock spec, tooling and verification are built once and then used by every run and every block. Fifth, the asymmetry is stark: a modest clock-methodology investment against respin-scale and schedule-scale exposure. So the pitch is simply that this protects the foundation of all timing signoff against systemic, silent clock errors, preventing both escapes and wasted closure.



In PD / signoff. This is what you say when clock infrastructure work has to be funded as a line item rather than absorbed. It also changes how you report status upward: a clean timing report from an unverified clock model is not evidence of a healthy block, so the signoff dashboard should show clock-verification gate status alongside slack, making the investment's value visible.

Trap. Selling it as engineering quality or best practice, which reads as optional polish under schedule pressure. The argument that actually lands is the asymmetry - modest, amortized, once-built cost against a likely, systemic, silent failure mode that can invalidate the tapeout decision itself.

Expect next. Why does a clock modelling error carry respin-scale risk rather than a localized timing risk?

Clocks / Generated Clocks

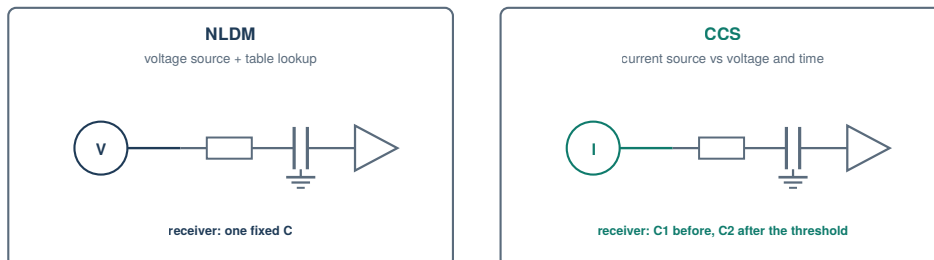
INTERMEDIATE

21 Compare NLDM and CCS for multivoltage scaling - which is the right choice and why?

CCS is the recommended model for scaling library groups: it represents the cell as a current source and captures nonlinear voltage dependence, whereas NLDM's voltage-independent delay and slope tables drift from SPICE across a wide voltage sweep.

NLDM describes a cell with delay and output-slope tables that are indexed by input slew and output load, but that carry no real voltage dependence of their own. Within a narrow voltage band that is fine. Stretch it across a wide sweep and the single-slope assumptions behind those tables progressively diverge from what SPICE says the cell actually does, because the shape of the driving waveform, not just its scale, changes with supply voltage. CCS takes a different approach: it models the cell as a current source, so the driver behaviour is described in a form that reproduces the nonlinear dependence on supply voltage much more faithfully. The recommendation to use CCS for scaling library groups follows directly from that. Multivoltage analysis is, by definition, the business of running the same cells at several different voltages and interpolating between characterized points - which is precisely the regime where NLDM's error is largest and where CCS earns its extra characterization and runtime cost. Put another way, the accuracy gap between the two models is smallest exactly where you do not need scaling and largest exactly where you do. If you are building voltage scaling library groups for a DVFS design, treating NLDM as good enough imports an error that grows with the very voltage range you added scaling to cover.

NLDM versus CCS driver models



A current-source driver keeps working when the load is strongly resistive, where a single voltage-table lookup starts to drift.

In PD / signoff. At library setup for any multivoltage or DVFS signoff, you check what the scaling library group is actually made of. If the vendor delivered only NLDM data, you know your interpolated corners carry model error that gets worse the further you are from a characterized voltage, and you either request CCS libraries or tighten the spacing of characterized voltage points. It also gates DVD analysis, which needs CCS noise data.

Trap. Assuming interpolation accuracy depends only on how many voltage points were characterized, and so building a scaling library group from NLDM data on the argument that the points are close together. The model form is the dominant error source across a wide sweep, not the point spacing.

Expect next. Why does modelling the driver as a current source rather than a delay table capture voltage dependence more accurately?

Multivoltage / General

ADVANCED

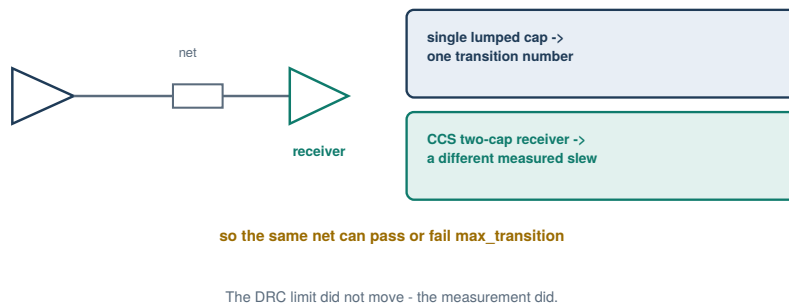


22 Explain the interaction between CCS receiver modeling and max-transition/max-capacitance DRC characterization.

CCS adds receiver_capacitance1/2 and driver tables, so safe max_capacitance is the lowest maximum load index across all of them and safe max_transition the lowest maximum slew index across NLDM, CCS driver and receiver_capacitance2. Otherwise DRC passes while delay calc extrapolates.

The purpose of the design-rule limits, from a delay-calculation point of view, is to guarantee that delay calculation never has to extrapolate beyond characterized data. CCS complicates that guarantee because it consults more tables than NLDM does: receiver_capacitance1 and receiver_capacitance2 tables for the load, plus driver tables for the slew, in addition to the NLDM tables. So the safe max_capacitance is the lowest maximum load index across all of them, and the safe max_transition is the lowest maximum slew index across the NLDM tables, the CCS driver tables and receiver_capacitance2. Set the limit from the NLDM table alone and you get a specific silent failure: a CCS table with a smaller max index will be extrapolated - raising RC-011 - on nets whose NLDM-based DRC check reports a clean pass. The companion setting closes the other half of the loop. With report_capacitance_use_ccs_receiver_model true, the max-capacitance checks reflect the CCS loads that delay calculation actually uses rather than the smaller lumped pin caps, so the check and the model agree on what the load is. The two features are coupled by design: correct DRC limits are what keep CCS delay calculation inside its own tables.

The receiver model changes the number the DRC is judged on



In PD / signoff. This belongs in library qualification, not in block-level timing. When a library is brought up, derive max_transition and max_capacitance from the minimum of the maxima across NLDM and CCS tables, and enable report_capacitance_use_ccs_receiver_model true in the signoff script. A passing DRC report is only meaningful if the limits were derived that way.

Trap. Deriving the DRC limits from the NLDM tables alone. If a CCS table has a smaller max index, delay calc extrapolates and raises RC-011 while max_transition and max_capacitance both report clean - so the design looks DRC-legal on nets whose delays are extrapolated and inaccurate.

Expect next. Why does max_transition include receiver_capacitance2 in the set of tables you take the minimum maximum slew index over?

Delay Calculation / CCS Modelling · report_capacitance_use_ccs_receiver_model

EXPERT

23 Explain why set_voltage's max-delay number is the lower voltage in a corner pair, connecting it to STA fundamentals.

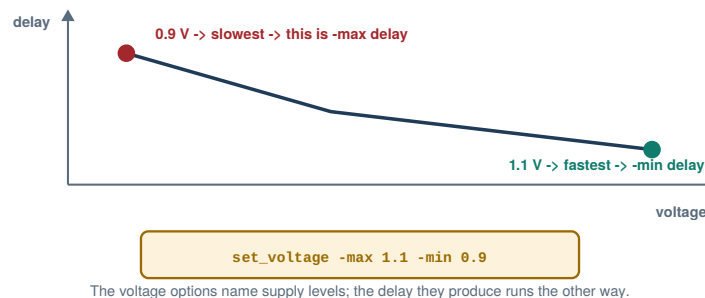
Max-delay means worst setup, worst setup means slowest cells, and cells are slowest at low voltage because there is less drive current. So the max-delay number is the low corner and the -min number is the high one.

The confusion comes from reading max-delay as max-voltage, and the fix is to walk the chain deliberately. Max-delay analysis exists to catch setup failures, and setup failures happen when cells are slow. Cells are slowest at low voltage, because lower supply means less drive current and therefore longer delays. So the voltage that produces max delay is the low corner. Run the same logic for the other side: min-delay analysis catches hold failures, hold failures happen when cells are fast, and cells are fastest at high voltage - so the min-delay voltage is the high corner. That is why an example like set_voltage 0.8 -min 1.2 VDD is correct even though it looks backwards, with the bare number smaller than the -min number. The bare number is the max-delay voltage, 0.8, the slow corner. The -min value, 1.2, is the min-delay voltage, the fast corner. The single sentence worth carrying into an interview is max-delay equals worst setup equals slow equals low voltage. Once you have that, the apparent inversion stops being something to memorize.



Note also what the mapping actually is: max-delay and min-delay map to slow and fast, not to numerically high and low. That distinction matters because inverted-temperature behavior or specific corner definitions can complicate which numeric value ends up where, while the delay-role mapping holds.

Max delay comes from the lower voltage



The voltage options name supply levels; the delay they produce runs the other way.

In PD / signoff. Every multivoltage `set_voltage` line you write depends on getting this straight, and it is the first thing to check when reviewing someone else's constraint file. If the bare value is the higher number, setup is being analyzed at the fast corner and hold at the slow one - both optimistic. A quick `report_timing -voltage` on a setup path should show the low corner voltage on the cells; if it shows the high one, the `set_voltage` arguments are swapped.

Trap. Reading max-delay as max-voltage and writing `set_voltage 1.2 -min 0.8 VDD`, which analyzes setup at the fast corner and hold at the slow corner. Nothing errors and both checks look comfortable, because each is being run at exactly the voltage that makes it easy to pass.

Expect next. Does that mapping always put the numerically lower voltage on the max-delay side, or can corner definitions and inverted-temperature behavior break the numeric pattern while the slow/fast roles hold?

Multivoltage / General · `set_voltage`, `report_timing`

ADVANCED

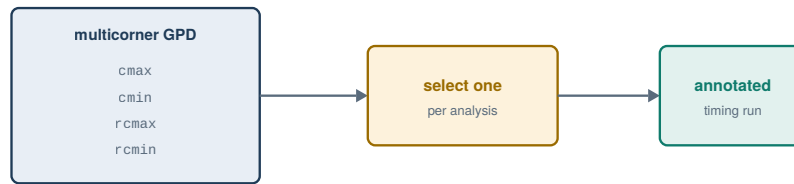
24 Explain the trip from a StarRC multicorners GPD to a single-corner PrimeTime annotation, and why multicorners extraction is worth it.

StarRC with `SIMULTANEOUS_MULTI_CORNER: YES` extracts several corners in one run into a multicorners GPD; PrimeTime picks one with `set_app_var parasitic_corner_name <corner>` then reads it. It pays off because the expensive geometric processing is done once for all corners.

On the extraction side, StarRC run with `SIMULTANEOUS_MULTI_CORNER: YES` extracts several parasitic corners at once and stores them together as a single multicorners GPD. On the PrimeTime side you choose which one you want with `set_app_var parasitic_corner_name <corner>` and then read it with `read_parasitics -format gpd -keep_capacitive_coupling <dir>`; the read report echoes the annotated corner name, which is your confirmation that you got the corner you asked for. The economics are what make it worth doing. MCM signoff needs timing at many RC corners - cbest, cworst, rcworst, typ and so on - and if you extract each corner in its own StarRC run you repeat the geometric processing every time, which is the expensive part of extraction. A single multicorners extraction does the geometry once and emits all the corners from it, which cuts extraction turnaround dramatically. PrimeTime then cheaply annotates whichever corner a given scenario needs out of that one database. If you enable the Parasitic Explorer before reading, you can even query across corners using `-parasitic_corners`. Structurally it is the parasitic-side analogue of cross-library scaling: do the costly work once and reuse it across corners rather than paying for it per corner.



Many extracted corners in, one annotated corner out



Extracting every corner once is cheaper than re-extracting, and it guarantees the corners come from the same run.

In PD / signoff. This shapes how the extraction request is written for an MCMM signoff: one multicorner StarRC run rather than one run per RC corner, with each PrimeTime scenario setting `parasitic_corner_name` to the corner it needs before reading the shared GPD. The corner echo in the read report is the artefact you check per scenario, since a scenario silently annotated at the wrong RC corner produces credible but wrong timing.

Trap. Reading the multicorner GPD without setting `parasitic_corner_name` for the scenario, or not checking the corner name echoed by the read report. The annotation succeeds and the timing looks reasonable, but the scenario is signed off against the wrong RC corner - a failure no completeness check will catch.

Expect next. Why does the geometric processing dominate the cost of running StarRC once per corner instead of once for all corners?

Back Annotation / SPEF & Parasitics · `set_app_var`, `read_parasitics`, `parasitic_corner_name`, `parasitic_corners`

INTERMEDIATE

25 Why should you prefer the `-from/-to` form of an exception over the `-through-plus-clock` form?

Because the special form - startpoint or endpoint given as `-through` with a clock as `-from/-to` - supports all valid start and end types but is more computationally intensive, especially on large designs. Plain `-from/-to` is cheaper.

There is a special way to write a path specification in which the startpoint or endpoint is given as a `-through` while a clock object is supplied to `-from` or `-to`. It exists for good reason: it supports all valid start and end types, so it can express specifications that the plain form cannot. But it is more computationally intensive to evaluate, and that cost grows with design size - on a large design it can be a meaningful part of runtime, because the tool has less structure to work with when narrowing the candidate path set. Plain `-from/-to` is cheaper, so the guidance is to use it whenever it can express what you mean, and reserve the special form for the cases that genuinely require it. This is the same efficiency instinct that shows up all over exception writing: several forms are functionally correct, and the senior answer is the cheapest correct one. It matters most when the specification lands in an exception rather than a one-off report, since an exception written in the expensive form is re-evaluated on every path search in every scenario, multiplying a small per-run cost across the whole mode and corner matrix.

In PD / signoff. Runtime tuning on a large SoC. When a signoff run is slower than expected, the exception file is worth scanning for specifications written in the expensive `-through-plus-clock` form where a plain `-from/-to` would say the same thing - the cost is paid in every scenario of the matrix, not once.

Trap. Reaching for the `-through-plus-clock` form by habit because it always works, without checking whether plain `-from/-to` expresses the same intent. Both give correct results, so nothing flags it - you just pay the extra path-search cost on every run across the whole scenario matrix.

Expect next. Which specifications genuinely require the `-through-plus-clock` form rather than plain `-from/-to`?

Timing Exceptions / Path Specification

ADVANCED

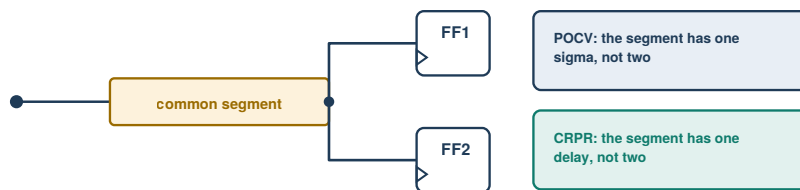
26 Explain the interaction between POCV and clock reconvergence pessimism removal on a clock path - do they double-count?

No - they correct different artifacts. CRPR removes the double-count on the clock segment shared by launch and capture; POCV graph-merging removal removes the stochastic-max inflation at converging gates. Both must be active on a clock path.



They address different pessimisms and are complementary rather than redundant. CRPR removes the shared-clock-segment double count that OCV and POCV introduce when the launch and capture clock paths overlap, since the analysis would otherwise charge the common segment two different delays at once. POCV graph merging pessimism removal removes the stochastic-max inflation at converging gates, where the statistical max of several fan-in distributions exceeds any single delay. On a clock path you can genuinely have both: the common clock portion needs CRPR to reconcile launch against capture, and any reconvergent clock structure, or converging data at the endpoint, needs graph-merging-pessimism removal. They cannot double-count because the mechanisms they correct are different - CRPR is about how a shared segment is treated on the launch side versus the capture side, while graph merging is about the statistical operation of merging fan-in distributions. There is one subtlety worth having ready: under POCV, the delays CRPR reconciles are themselves distributions with a mean and a sigma, so CRPR operates statistically too. The common path pessimism is computed on the distribution, and the clock reconvergence pessimism line coexists with the statistical graph pessimism line in a POCV `report_timing -variation`. An accurate POCV clock path needs both corrections enabled and understood; a candidate who conflates them - thinking graph-merging removal subsumes CRPR, or the reverse - misses that they fix orthogonal artifacts.

POCV and CRPR act on the same shared clock segment



same principle, different quantity - so they do not double-count

One removes duplicated variation, the other duplicated delay; each is applied once.

In PD / signoff. On a POCV clock path debug you open `report_timing -variation` and look for both lines: the clock reconvergence pessimism entry and the statistical graph pessimism adjustment. One missing line tells you which correction is disabled or not applying, and that is usually why a deep clock path looks unaccountably pessimistic.

Trap. Assuming that because POCV already removes merging pessimism, CRPR is redundant and can be relaxed - or the reverse, that CRPR covers the convergence artifacts so the statistical adjustment can be ignored. Disabling either leaves a real artifact in place and inflates clock path pessimism.

Expect next. Under POCV, what exactly is the common path pessimism computed on when the shared clock segment's delay is a distribution?

Variation / POCV Distributions · `report_timing`

EXPERT

27 A path shows negative setup slack, but you believe it's a genuine multicycle path that was never constrained. Give the end-to-end procedure to confirm and fix it safely.

Confirm the path exists and see what constrains it today, get the real cycle count from the designer, apply `set_multicycle_path -setup N` with the matching `-hold`, then re-verify with `report_timing -exceptions all` that it is dominant on exactly that path.

Treat it as an investigation rather than a fix. Start by confirming the logical relationship: `all_fanout -from <launch> -to <capture>` verifies that the path exists the way you believe it does. Then inspect what constrains it today with `report_timing -from <launch> -to <capture> -exceptions all` - is some exception already dominant, or is the path genuinely unconstrained beyond the default single-cycle check? Next comes the step people skip: confirm the hardware intent with the designer. How many cycles does the enable actually allow? Never invent a multicycle number to make timing pass. With that number in hand, apply `set_multicycle_path -setup N` together with the matching `-hold` of N-1, or whatever hold value the real enable behaviour dictates. Re-run `report_timing -exceptions all` and confirm two things: the multicycle exception is now dominant on exactly this path, and nothing unintended was swept in by your object specification. Check the hold side explicitly - avoiding a bogus hold requirement is the entire purpose of the `-hold` option. Finally, diff the constraint against the golden SDC and document the rationale. The discipline behind every step is that a multicycle path is a design fact, verified with the designer, not a slack-closing trick, and an interviewer is watching to see which of those two you believe.



In PD / signoff. This is the standard route for any exception added during timing closure. The artefacts you leave behind matter as much as the constraint itself: the designer's confirmation of the cycle count, the before-and-after `report_timing -exceptions all` output, and the SDC diff. Reviewers approve the exception on that evidence, not on the improved slack number.

Trap. Applying `-setup N` because the slack happens to fit in N cycles, skipping both the designer confirmation and the hold check. You get a green setup report, an invented hold requirement that gets buffered away, and a functional failure in silicon if the enable never actually allowed N cycles.

Expect next. What exactly would you ask the designer to establish the cycle count, and which clock's cycles are you counting?

Timing Exceptions / Multicycle Paths · `report_timing`, `all_fanout`, `set_multicycle_path`

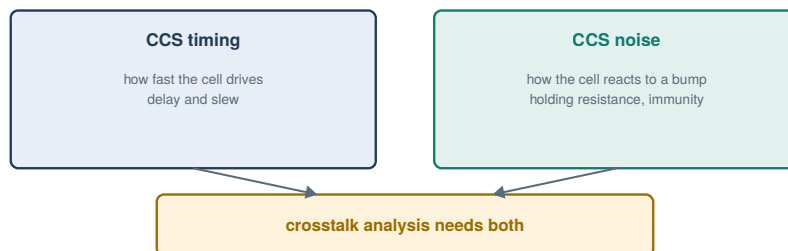
ADVANCED

28 Why must all libraries in the scaling groups have CCS noise data specifically, beyond CCS timing?

Because SMVA is a full signoff feature that includes noise across voltage combinations, and voltage-dependent noise response - crosstalk bumps and immunity - is captured only in the CCS noise models. CCS timing alone cannot produce it.

The reason comes down to what each model contains. CCS timing gives you delay behaviour, essentially how a cell drives a load. CCS noise gives you something different: the current-versus-disturbance waveform models that describe how a cell responds to and propagates a noise event. Crosstalk bumps and noise immunity are voltage-dependent phenomena, and their voltage dependence lives in those noise models, not in the delay tables. SMVA is not a timing-only feature. It is a full signoff capability, and that includes noise and SI analysis across voltage combinations - `update_noise` and `report_noise` are scenario-aware precisely because noise is analyzed per DVFS scenario. So the tool has to evaluate each cell's noise response at each voltage it might be operating at, which it can only do by scaling between library characterisations that actually carry noise data. That is why the requirement lands on every library in the scaling groups rather than most of them. A scaling group's job is to interpolate cell behaviour across voltage; if one member library lacks CCS noise data, there is nothing to interpolate for that portion of the voltage range and the noise computation across voltages cannot be performed. The distinction to hold onto for the interview is that having CCS libraries is not the same as having CCS noise libraries - the requirement is specifically about the noise models, beyond CCS timing.

CCS timing and CCS noise answer different questions



A scaling group with timing data but no noise data cannot answer the SI half at the scaled voltage.

In PD / signoff. This turns into a library-qualification gate before an SMVA run. When you assemble `define_scaling_lib_group` for each Vt flavour, verify CCS noise data is present in every member library at every corner, not just in the nominal one. A library set that passes a timing-only sanity check will still block the noise half of SMVA signoff.

Trap. Assuming CCS-qualified libraries are automatically adequate for SMVA. CCS timing satisfies delay scaling but not noise, so a scaling group missing CCS noise data in even one library leaves the voltage-dependent noise computation impossible - discovered late, when the noise stage of signoff is reached.

Expect next. What specifically can CCS noise models represent about a cell's behaviour that CCS timing models cannot?

SMVA / Rail Voltages · `update_noise`, `report_noise`, `define_scaling_lib_group`

INTERMEDIATE

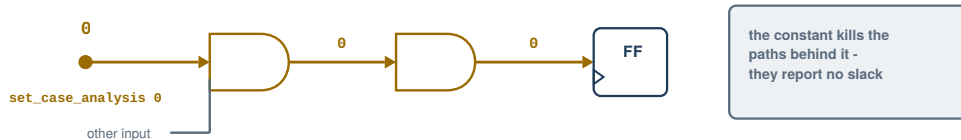
29 Why is "case analysis propagates forward only" a fundamental limitation to keep in mind?

Setting a constant constrains only what is downstream of it - PrimeTime never infers the upstream values that would be logically necessary to produce it - so you must set the constant where the value can propagate forward to where it is needed.



Case analysis is a forward-only operation, and that shapes where you are allowed to put constants. When you pin a value on a pin, PrimeTime constrains only what lies downstream of that pin; it will not back-solve the upstream logic to work out which input combinations would have produced the constant. So if the mode you want to force is determined by an internal signal, you have to place the case analysis at a point from which the value can actually propagate forward to the arcs and conditions you care about. Pinning a downstream pin does not reach backwards to constrain the logic that feeds it. This is precisely why mode-select constants belong on the controlling input - the RAM's `read_write` pin, for instance - and not on some internal node you merely wish were constant. The practical failure mode is quiet: you set a constant somewhere convenient, nothing errors, and the mode condition upstream is never evaluated against the value you intended, so the mode never resolves the way you assumed. Framed as a rule of thumb, always ask which pins the value needs to reach, then walk backwards through the logic yourself to find the real control point, and set the constant there. The tool will do the forward half; the backward reasoning is yours.

How a case analysis constant propagates forward



The value propagates as far as the logic lets it, so one setting can silence a large cone of the design - which is exactly why coverage should be checked after applying it.

In PD / signoff. When bringing up constraints for a new block, an engineer traces each mode or configuration requirement back to a real control port and sets the constant there, then verifies with a fanin trace that the intended pins received the value. It is the difference between a mode setup that takes effect and one that silently leaves the design in its default all-enabled state.

Trap. Pinning an internal node deep in the logic because that is where you want the value, expecting the tool to back-solve the upstream inputs. It will not, so the mode-selection condition is never evaluated as false and the intended masking never happens.

Expect next. How would you locate the correct forward-propagating control point when the mode is determined by a register you cannot reach from any port?

Case & Mode Analysis / Mode Analysis · `read_write`

ADVANCED

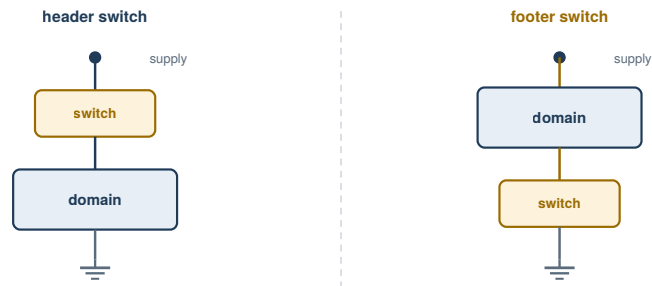
30 A path in a switchable domain reports the wrong voltage after a power switch.

Work outward from the switch: `report_power_switch` for the On State, `report_power_network` for the topology through it, `report_supply_net` on the output net for its voltage states, then `report_timing -voltage` or `pg_pin_info`'s `voltage_source` on the cells.

Trace it in the same order the tool itself resolves voltage. Start with `report_power_switch` and read the switch's On State expression, confirming that the control conditions you believe are active really do put it in the pass state you expect - if they do not, everything downstream is explained already. Next run `report_power_network` to see the actual net topology through the switch, which tells you whether the path from the input port to the cells you care about exists at all and goes where you think. Then `report_supply_net` on the switch's output net to inspect its voltage states and ranges. Finally, look at the cells themselves with `report_timing -voltage`, or read `voltage_source` through `pg_pin_info`, to see what voltage actually propagated to their PG pins. That last step distinguishes a wrong voltage from a fallback voltage. The single most common root cause in this scenario is that the switch's output net never had a `set_voltage` applied to it - the switch is fine, the topology is fine, but the terminating net carries no voltage, so the walk cannot resolve one and you get the UPF-029 message. Fix the voltage on the output net rather than hunting for a connectivity bug.



Header and footer power switches



A header interrupts the power side and a footer the ground side, which is why the two take different external-net options.

In PD / signoff. This is the standard debug ladder when a switchable-domain block reports delays that do not match its intended DVFS point. Each step narrows the failure to one layer - switch state, topology, net voltage, pin voltage - so you stop guessing. In a signoff script, checking that every switch output net has a `set_voltage` is worth doing pre-emptively before timing is released.

Trap. Assuming the switch or the connectivity is broken and rewriting UPF, when the switch output net simply has no `set_voltage` on it. That case reports as UPF-029 and shows up as PG pins whose `voltage_source` is not the net you expected, not as a missing connection.

Expect next. What does UPF-029 tell you about the switch's output net, and which command fixes it?

Multivoltage / UPF & Power Domains · `report_power_switch`, `report_power_network`, `report_supply_net`, `report_timing`

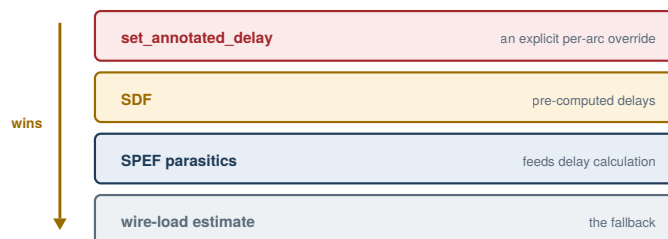
EXPERT

31 Construct a scenario where the back-annotation precedence order produces a subtly wrong result that passes all obvious checks.

A forgotten `set_resistance 5 [get_nets long_route]` is tier 2 and outranks tier-3 detailed parasitics, so PrimeTime uses the stale lumped 5 ohms while `report_annotated_parasitics -check` still passes because the SPEF data is present, just shadowed.

Picture this. Months ago someone added `set_resistance 5 [get_nets long_route]` to a shared constraint file to model a route before extraction existed, and nobody removed it. Later the team reads an accurate full-chip SPEF and runs `report_annotated_parasitics -check`, which passes - the net does have a complete RC network from SPEF. Everyone believes `long_route` uses the extracted resistance. But `set_resistance` sits at tier 2 and outranks the tier-3 detailed parasitics, so PrimeTime uses the lumped 5 ohms and silently ignores the SPEF resistance for that net. The completeness check passes, timing looks plausible, and nothing errors, yet `long_route` delay is computed from a stale hand-set value rather than the real extraction, which can either mask a genuine violation or manufacture a fake one. It survives every obvious check because the failure mode is not missing data - it is shadowed data, and `-check` does not flag shadowing. Catching it requires a deliberate shadowing audit: cross-reference `report_annotated_delay` and the lumped-override reports against the nets you expect the SPEF to own, or `report_net` on the suspect net and notice that its resistance does not match the SPEF. That is exactly why verifying precedence shadowing is a distinct signoff gate from verifying completeness.

Which annotation wins when two sources cover one arc



The danger is a stale explicit override quietly outranking the fresh SPEF you just read.



In PD / signoff. At signoff, after reading the SPEF, an engineer diffs the list of nets carrying lumped R or C overrides against the nets the extraction is supposed to own, and runs `report_net` on suspicious long routes. Any hit means the constraint file is edited and timing rerun, because otherwise the ECOs generated from that run are fixing the wrong net.

Trap. Believing a passing `report_annotated_parasitics -check` means the extracted values are in use. Completeness and precedence are separate properties: the check confirms an RC network exists on the net, not that the tool chose it over a higher-tier lumped override.

Expect next. How would you script an automated shadowing audit that flags every net where a lumped override outranks available detailed parasitics?

Back Annotation / SPEF & Parasitics · `set_resistance`, `get_nets`, `report_annotated_parasitics`, `report_annotated_delay`

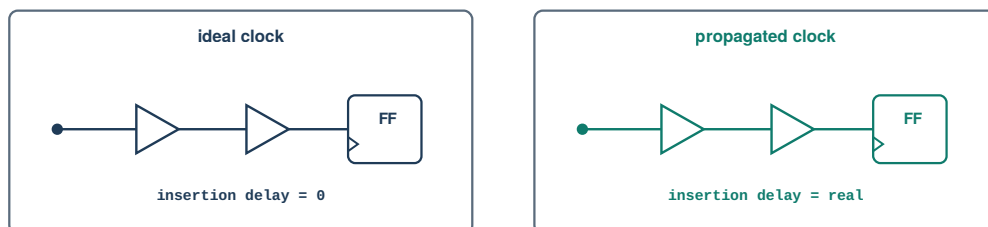
ADVANCED

32 A team proposes leaving clocks ideal (skipping propagated timing) post-CTS to save runtime near a deadline. How do you respond?

Refuse it. Ideal clocks post-CTS ignore the real clock-tree latency and skew, so they silently mis-time every path in the domain - especially hold, which is skew-dominated. Solve runtime with parallelism and resources instead.

Name the risk first. Ideal clocks use estimated or zero network latency plus estimated uncertainty, not the real skew of the built tree. Post-CTS that skew exists and can be significant, and ideal analysis simply cannot see it, so a path that genuinely fails because of real skew shows no violation. That is a missed violation, and a missed violation at signoff is a silicon escape. The exposure is worst for hold, because hold is dominated by the skew between launch and capture edges: a small real skew can create or cure a hold failure, and hold failures are often catastrophic and unfixable post-silicon. Second, ideal is not a cheaper flavour of accurate. The whole reason you propagate after CTS is that the real tree's latency and skew materially change the timing; skipping propagation makes the signoff a statement about a design that was never built. Third, runtime is not the real trade being offered. The saved hours buy a meaningless signoff. If runtime is genuinely the constraint, attack it directly - threaded multicore analysis with `set_host_options -max_cores`, adequate memory and cores, high-capacity mode when memory-bound, DMSA for multi-scenario runs, and removing redundant updates and wasteful reporting. If the full signoff truly cannot finish, prioritize the highest-risk corners, modes and blocks rather than degrading accuracy everywhere. Never falsify the clock model to make the clock on the wall look better.

Ideal versus propagated clock



An ideal clock hides the tree so pre-CTS numbers stay comparable; a propagated clock uses the real buffer delays, which is the only mode that can expose skew and hold problems.

In PD / signoff. This is a signoff gate decision. Before releasing timing you confirm every clock is propagated, not ideal, and you fail the run if any is not. When the schedule squeezes, the lever you pull is `set_host_options -max_cores`, more machine, high-capacity mode or DMSA - and if something still has to give, it is corner coverage order, chosen deliberately, not clock accuracy.

Trap. Assuming ideal clocks are merely pessimistic, so a clean ideal run must be safe. For hold they are optimistic, because the real skew that causes the race is absent from the model. The run passes, looks tidy, and hides exactly the failure mode you cannot fix after tapeout.

Expect next. Why is ideal-clock signoff especially dangerous for hold checks?

Clocks / Uncertainty & Jitter · `set_host_options`

EXPERT



33 Why does constraint variation (setup and hold sigma) need its own enable on top of POCV, and what goes wrong if you forget it?

POCV models delay variation by default but not the setup and hold values themselves, because that needs LVF constraint-sigma data and its own cost. Forget `timing_enable_constraint_variation` and you compare a distributed arrival against a deterministic requirement - a silent optimism.

By default POCV models variation on delays, both data and clock path delays, but not automatically on the setup and hold constraint values. Those come from the library's constraint arcs, and the variation of the requirement itself - the sigma of the setup or hold number - needs LVF constraint-variation data plus an explicit `timing_enable_constraint_variation true`, with `timing_use_slew_variation_in_constraint_arcs` to include the slew-dependent part. It is a separate enable for two reasons: not all libraries carry LVF constraint sigma tables, since a library may have delay sigma without constraint sigma, and constraint variation is a distinct modeling layer with its own accuracy and runtime cost. The risk of forgetting it is that the setup and hold requirements are treated as fixed, zero-variation numbers while the delays carry full distributions - an inconsistent and optimistic model. Physically, a flop's setup and hold times also vary with process, because they are device characteristics, so ignoring their sigma under-margins the check: you are comparing a distributed arrival against a deterministic requirement and missing the requirement's own spread. At advanced nodes, where constraint variation is nontrivial, that can mask real setup and hold violations because the corner value of the check is too tight. For a rigorous POCV signoff you enable constraint variation, with the LVF data present, so both sides of the setup and hold inequality carry their true statistics. Forgetting it is subtle and silent - the analysis looks fully statistical but is only half statistical. Verify that the LVF constraint sigma tables exist, using `report_ocvm` on constraint arcs, and that the enable is actually set.

Half a statistical check is not a statistical check



`timing_enable_constraint_variation` is what puts a spread on the requirement side too.

In PD / signoff. Add both the enable and its data check to the POCV signoff gate list: `report_ocvm` on constraint arcs to prove the LVF constraint sigma tables exist, and a grep for `timing_enable_constraint_variation` in the setup. At advanced nodes this is where a chunk of hold margin lives, so an unset enable is a real signoff finding, not a nitpick.

Trap. Assuming that enabling POCV makes the analysis fully statistical. Delay sigma comes on, constraint sigma does not, so the check compares a distribution against a fixed number and the reported corner is optimistically tight - and the report gives no hint that half the inequality is deterministic.

Expect next. If a library has delay sigma but no constraint sigma tables, what value does the check use for the requirement and what does that do to your margin claim?

Variation / POCV Distributions · `report_ocvm`, `timing_enable_constraint_variation`, `timing_use_slew_variation_in_constraint_arcs`

ADVANCED

34 How do generated clocks interact with source latency, and how does edge-specific source latency propagation work?

A generated clock's source latency is the master's insertion delay from its origin to the generated clock's source pin. PrimeTime auto-computes it when the master is propagated, and it tracks the master's latencies per edge.

Think of a generated clock's source latency as inherited, not independently specified. It is the latency from the master clock's origin down to the generated clock's source pin - in other words the master's insertion delay up to the generation point. If the master clock has propagated latency, PrimeTime can compute this automatically: it propagates the master's real latency to the generated clock's source pin, so the generated clock inherits the correct upstream insertion delay and you do not specify it separately. If the master is ideal there is no real latency to propagate, so you supply the generated clock's source latency explicitly or it comes from the master's specified latency as applicable. On top of that, PrimeTime supports generated-clock edge-specific source latency propagation. The master's rising and falling edges, and their early and late values, may not reach the source pin with the same delay, because the clock



network's rise and fall delays through cells and nets can be asymmetric. Since a generated clock is derived from specific master edges through the divide, multiply or `-edges` relationship, which master edges it uses determines which source latencies it inherits. PrimeTime therefore tracks source latency per edge of the generated clock and propagates the master's edge-specific latencies through the generation relationship, rather than collapsing them into one averaged number. That is what makes generated-clock timing accurate where master-edge latency asymmetry is significant.

Source latency versus network latency



Source latency models delay before the definition point and applies to every clock edge equally. Network latency models the on-chip tree and is replaced by real delay once the clock is propagated.

In PD / signoff. This is why you propagate the master before trusting a divided clock's numbers post-CTS. In practice you define the generated clock with `create_generated_clock`, apply `set_propagated_clock`, and then read `report_clock` to confirm the source latency was inherited rather than left at an estimate - the difference shows up directly in launch and capture arrivals on paths across the divider.

Trap. Hand-specifying a source latency on the generated clock while the master is already propagated. You then override the real inherited insertion delay with a stale estimate, and because the command succeeds silently the generated domain times against a clock origin that does not match the built tree.

Expect next. Under what condition can PrimeTime auto-compute a generated clock's source latency?

Clocks / Generated Clocks · `create_generated_clock`, `set_propagated_clock`, `report_clock`

INTERMEDIATE

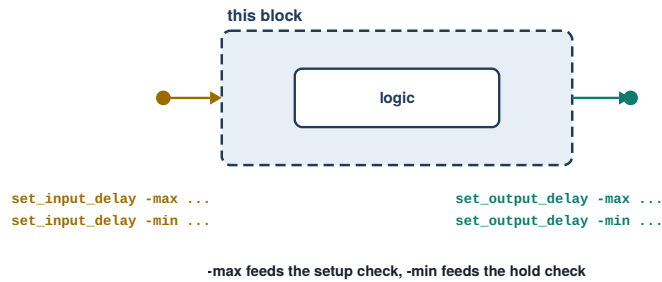
35 How would you constrain a design for both setup and hold analysis at the boundary?

Specify both `-max` and `-min` input and output delays at every boundary port. Setup analysis uses the maximum late external delays and hold uses the minimum early ones, so constraining only one leaves the other check unbudgeted.

Setup and hold are separate checks that consume different extremes of the external delay range, so the boundary has to supply both. On inputs, `set_input_delay -max` gives the maximum external delay, which feeds the setup check on the internal path starting at that input, and `-min` gives the minimum external delay, which feeds the hold check. A single value typically applies to both, but for accurate analysis you set the max and min external delays separately, because the real external path has a range and the two values genuinely differ. On outputs, `set_output_delay -max` gives the longest external path plus the external register's setup, which feeds the setup check on the internal path ending at that output, and `-min` gives the shortest external path minus hold, feeding the hold check. Drive and load are set for the min and max corners as appropriate. The reasoning is that setup checks whether the latest-arriving data still meets the capture register's setup requirement, so it uses the maximum delays throughout, while hold checks the earliest arrival against the hold requirement and uses the minimum delays. The completeness consequence is the part people miss: both min and max at every boundary port, or the setup or hold check on those paths is incomplete. Set only `-max` and the hold paths from and to that port have no early external budget, so they are effectively unconstrained for hold.



A boundary needs a max and a min number at both ends



In PD / signoff. This is a boundary-completeness audit you run before hold closure, typically right after CTS when hold becomes the live problem. You confirm from `report_port` that every port has both a min and a max delay in each applicable direction, because a max-only interface will report clean hold at the boundary regardless of what the silicon does.

Trap. Specifying only `-max` delays because setup is what everyone is watching. The hold paths from and to those ports then carry no early external budget and are silently unchecked, so a real hold violation at the interface never appears - and hold failures are the ones you cannot work around after fabrication.

Expect next. Which delay, min or max, feeds the setup check, and which feeds hold?

Constraints / I/O Timing · `set_input_delay`, `set_output_delay`, `report_port`

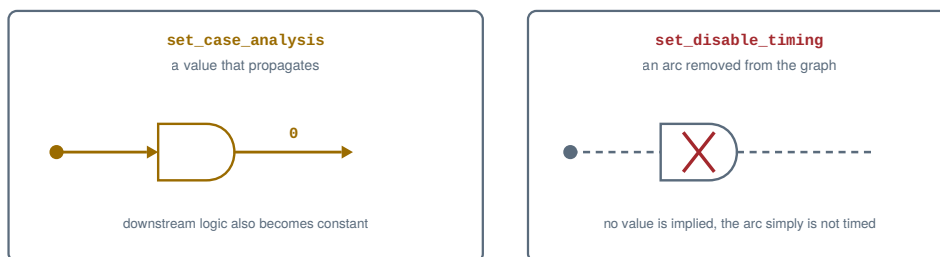
ADVANCED

36 Compare disabling a path with case analysis, `set_disable_timing` and `set_false_path` - mechanism, blast radius, and when each is the right tool.

Case analysis sets a value and structurally kills whatever arcs that value controls; `set_disable_timing` removes a specific arc or object; `set_false_path` removes exactly the specified point-to-point paths - and the efficiency ladder prefers them in that order.

All three remove paths, but they operate at different levels. Case analysis sets a value, and the propagated constant disables whatever arcs that value structurally kills, so its blast radius is everything the constant controls - a whole mode cone from a single signal. It is right when a real control signal or mode is the root cause. `set_disable_timing` removes a specific arc, pin, or cell from analysis directly, so its blast radius is every path through that object, and it is right when all paths through a specific point are irrelevant regardless of logic - a dead pin, an unused arc. `set_false_path` is a point-to-point exception whose blast radius is exactly the specified paths; it is the most flexible but it is tracked as an exception with the usual precedence bookkeeping, and it is the costliest at scale. That gives an efficiency ladder: prefer case analysis first because it addresses the root cause structurally with zero exception bookkeeping, then clock groups or `set_disable_timing`, and false paths last. Applied to concrete cases, a mode is a job for case analysis - one signal, the whole cone; a dead arc is a job for `set_disable_timing`; and a specific crossing that is not mode-governed is a job for a false path. Matching the tool's natural granularity to your intent is the whole game, which is the same lesson as choosing the smallest highest-level object for an exception, or matching the parasitic tool to the net.

set_case_analysis versus set_disable_timing



Reach for case analysis when a pin really is tied; reach for `disable_timing` when an arc must not be traversed.



In PD / signoff. Every time an engineer is about to add an exception, this ladder decides which command to reach for. Choosing case analysis where a real mode signal exists keeps the exception list short, which directly reduces the exception-precedence debugging and the runtime cost that a large false-path collection imposes on every timing update in the ECO loop.

Trap. Reaching for `set_false_path` because it is the most flexible. Doing that for a mode-governed cone replaces one structural constant with many tracked exceptions, adding precedence bookkeeping and runtime cost while leaving the actual root-cause control signal unconstrained.

Expect next. How would you decide between `set_disable_timing` and a false path for a crossing that is only sometimes mode-governed?

Case & Mode Analysis / Mode Analysis · `set_disable_timing`, `set_false_path`, `disable_timing`

EXPERT

37 How would you decide which clock-related decisions to standardize versus leave to block owners?

Standardize whatever must be chip-consistent to compose - primary clock definitions, the relationship framework, the modelling methodology, the single-source discipline and verification gates - and delegate block-internal generated clocks, gating details and margin refinements within that framework.

The dividing line is whether a decision is chip-shared and has to compose, or purely block-internal. Standardize the chip-level clock definitions - the primary clocks, their periods and waveforms, and the PLL and reference scheme - because these are shared across blocks and must be identical everywhere they appear. Standardize the clock-relationship framework, meaning the chip-level clock groups declaring which domains are synchronous, asynchronous or exclusive, because crossings span blocks and a crossing cannot be asynchronous in one block's view and synchronous in another's. Standardize the clock-modelling methodology as well: ideal versus propagated per stage, uncertainty conventions, jitter as dynamic latency, and generated-clock conventions. Standardize the single clock-constraint source discipline and the verification gates, `check_timing` clean of unlocked registers and `report_clock`. Those five things are what make the clocks consistent and composable across the chip. Then delegate the rest to the people with the design knowledge: block-internal generated clocks, such as a divider feeding only that block's logic, defined per the standard conventions; block-internal clock-gating details and their checks; and block-specific uncertainty refinements tuned within the chip-level framework, for example a residual based on that block's own clock tree. None of those cross a boundary, so provided they conform to the standardized definitions and relationships they cannot damage chip-level consistency. The result is block-owner autonomy inside a consistent chip clock framework.

In PD / signoff. In practice this is how you write the constraint template. The shared file the block sources contains the primary clocks, the group matrix and the modelling conventions and is read-only to block owners; the block's own file holds its internal `create_generated_clock` calls, gating checks and residual margin. Review effort then concentrates on the shared file and on whether each block's additions conform.

Trap. Standardizing too far and freezing block-internal choices, or too little and leaving the relationship matrix to blocks. The second is the damaging one: independently invented chip-level clocks and crossing declarations conflict only at integration, and they conflict silently, because every block's own run looked clean.

Expect next. Which clock decisions are legitimately block-internal, and what makes them safe to delegate?

Clocks / Generated Clocks · `check_timing`, `report_clock`, `create_generated_clock`

ADVANCED

38 Explain why "only internal nets are annotated in SDF" and how that shapes your verification.

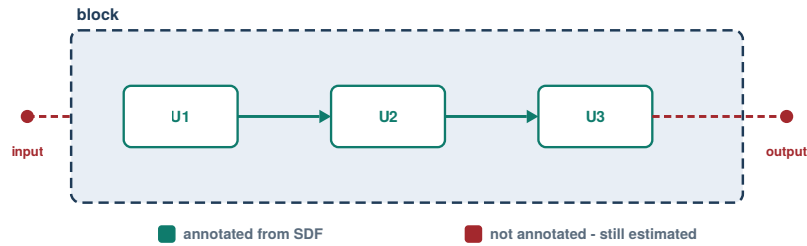
The SDF standard defines INTERCONNECT delays for nets internal to the design; nets on primary I/O ports belong to the environment and are modelled with `set_input_delay` and `set_output_delay`. So non-annotation of primary-port nets is correct, not a gap.

The rule comes from the SDF standard, which defines INTERCONNECT delays for nets internal to the design. A net connected directly to a primary input or output port describes something outside the design - the environment's wiring and driver - and that is not the SDF's job to state. It is modelled instead by the boundary constraints, `set_input_delay` and `set_output_delay`. PrimeTime follows the standard, so it annotates delays on internal nets and does not annotate primary-input or primary-output nets. That has a direct consequence for how you read a coverage report. When `report_annotated_delay` lists net arcs from primary inputs and net arcs to primary outputs as not annotated, that is correct and expected behaviour, and chasing those entries as missing annotations wastes time and, worse, trains you to ignore the columns that matter. Your verification should focus on the internal net arcs and the



cell arcs columns. A nonzero Not Annotated count there is the real signal of a problem - typically a name mismatch between the SDF and the design, or genuinely missing SDF coverage. So the useful discipline is to know in advance which categories should be empty and which should not, and only treat departures from that expectation as findings.

SDF annotates what is inside, not what crosses the boundary



Boundary nets keep whatever the constraints gave them, so check them separately.

In PD / signoff. This changes what you believe when you read `report_annotated_delay` on an SDF-annotated session. You audit the internal net arcs and cell arcs columns and require them fully annotated, while accepting the primary-port categories as non-annotated by design. Getting this right is what keeps a coverage review honest, since a reviewer who cannot tell expected non-annotation from a real gap ends up accepting both.

Trap. Reading every Not Annotated entry as a defect and hunting the primary-port net arcs, or the reverse - being so used to non-annotated entries in the report that a real gap in the internal net arcs or cell arcs columns, from a name mismatch or missing SDF coverage, gets waved through.

Expect next. If the environment's delay on a primary-input net is not in the SDF, what is actually modelling it in your timing session?

Back Annotation / SDF Annotation · `set_input_delay`, `set_output_delay`, `report_annotated_delay`

EXPERT

39 Two engineers get different slack on the same path, same corner. One used `report_timing`, the other `report_crpr` plus hand arithmetic. How do you adjudicate?

The gap is almost certainly CRP merging. `report_timing` merges adjacent common-clock points whose CRP differs by less than `timing_crpr_threshold_ps`, default 5 ps, slightly under-crediting CRP. The `report_crpr` number is unmerged and more accurate.

Before you suspect either engineer, suspect the merging threshold. To save runtime, `report_timing` merges adjacent common-clock points whose CRP values differ by less than `timing_crpr_threshold_ps`, whose default is 5 ps. Merging coarsens the common-point search, which slightly reduces the CRP credited into slack, so the `report_timing` number comes out a little worse. The `report_crpr` accounting is unmerged, so hand arithmetic built on it lands on a slightly better and more accurate slack. Adjudicate in three steps. First, read the current `timing_crpr_threshold_ps`: if it holds a large design-phase value rather than the default, `report_timing` is under-crediting CRP by a meaningful amount and that alone explains the discrepancy. Second, lower it toward a signoff value and re-run `report_timing`, watching the slack converge upward toward the `report_crpr`-derived number - that convergence is the proof that merging, not a methodology difference, caused the gap. Third, for the official signoff number, prefer the unmerged `report_crpr` accounting, since it is the more accurate of the two. Then document the threshold you used, because it materially affects reported slack on paths through deep clock trees, and a number quoted without its threshold is not reproducible. The engineers were both right; they were reading two different accuracy settings of the same computation.



Same path, same corner, two different slacks



neither is wrong - they answer different questions

Before calling a difference a bug, check which analysis mode each engineer ran.

In PD / signoff. This comes up whenever two people quote different slack for the same path at signoff review, or when a block owner's number disagrees with the top-level regression. You read the threshold out of each run, re-run with it lowered, and record the value alongside the slack in the signoff document so the number can be reproduced later.

Trap. Assuming the two reports must agree and hunting for a constraint or corner difference, when the merging threshold explains it. The costlier error is quoting a signoff slack without recording `timing_crpr_threshold_ps`, since on deep clock trees a design-phase threshold can shift the number enough to change a pass into a fail.

Expect next. How would you pick a defensible signoff value for `timing_crpr_threshold_ps` on a given clock tree?

Operating Conditions / PVT & Corners · `report_timing`, `report_crpr`, `report_crprderived`, `timing_crpr_threshold_ps`

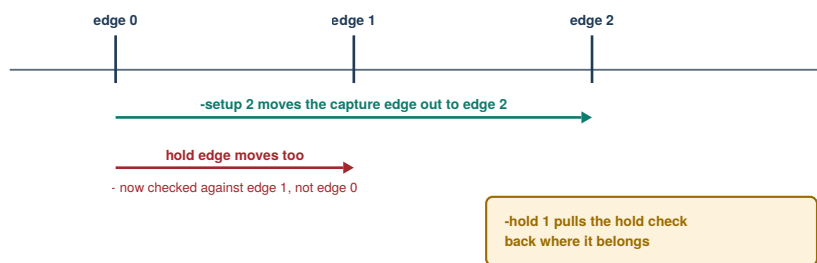
ADVANCED

40 Why does setting only `-setup 2` create a hold problem? Walk the edges.

Because hold checks are derived from the valid setup relationship. Moving the setup capture edge from $t=10$ to $t=20$ drags the implied hold check from $t=0$ to $t=10$, demanding a full extra cycle of data hold the hardware never needs.

Hold checks in PrimeTime are not defined independently - all hold relationships are derived from the valid setup relationship on the same path. Follow the edges on a same-clock path with a 10 unit period. By default, data launched at $t=0$ is captured by the very next capture edge at $t=10$; that is the setup relationship, and the matching hold check uses the capture edge one cycle earlier, at $t=0$, which is the ordinary same-edge hold check. Now write `-setup 2`. The setup capture edge moves out from the first edge at $t=10$ to the second edge at $t=20$, which is exactly what you wanted: the path gets two cycles to complete. But the implied hold check follows it, landing on the capture edge one cycle before the new setup edge, so it moves from $t=0$ to $t=10$. PrimeTime is now demanding that the data launched at $t=0$ still be stable at the capture flop at $t=10$, a full cycle later. Real hardware on a multicycle path almost never has that requirement - the enable that gated the two-cycle transfer does not also hold the source data steady for a cycle. So you get a large, artificial hold violation that hold buffering cannot honestly fix, which is why `-setup N` should always be written together with the matching `-hold` value that pulls that edge back where it belongs.

Why a multicycle setup needs a matching hold



hold cycles = (setup value) - 1 - (hold value)

Set `-setup N` and you almost always want `-hold (N-1)` as well, or the hold check tightens by $N-1$ cycles for free.



In PD / signoff. You meet this the first time you drop a `-setup 2` into an SDC to close a slow bus and the hold report explodes on endpoints that were clean the run before. Before accepting any hold-fixing ECO, look at the min-delay report on those endpoints and confirm the hold check is against the intended launch edge; if it moved out a cycle, fix the exception rather than inserting delay cells.

Trap. Writing `-setup N` alone and then closing the resulting hold violations with buffers. You are padding the design to satisfy a hold requirement the exception invented, spending area and power on it, and hiding the fact that the multicycle constraint is only half written.

Expect next. What `-hold` value restores the same-edge hold check once you have written `-setup 2`, and how does the cycle formula give it?

Timing Exceptions / General

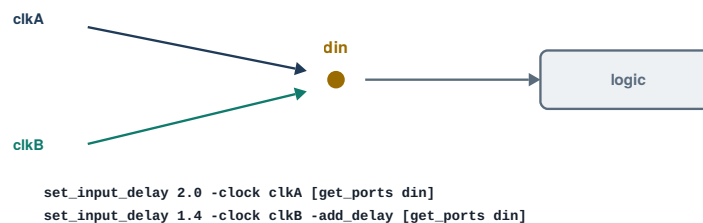
INTERMEDIATE

41 How do you constrain a port that receives paths from multiple clocks, and why is `-add_delay` needed?

Specify each clock's delay on the port with `-add_delay`. Without it, a new `set_input_delay` or `set_output_delay` removes the existing delay data on that port and replaces it, so the second command would wipe out the first clock's budget.

When paths from more than one clock, or from more than one edge, arrive at the same port, each needs its own budget on that port - and the mechanism for accumulating them is `-add_delay`. The reason it is mandatory rather than stylistic is the default behaviour: without `-add_delay`, a new `set_input_delay` or `set_output_delay` on a port removes the existing delay data there and replaces it. So specifying a second clock's delay without the option does not add a second budget, it silently discards the first. With `-add_delay`, the new delay sits alongside the existing ones and the port ends up constrained relative to all the relevant clocks. For example, `set_input_delay 4.5 -clock PHI1 {IN1}` followed by `set_input_delay 2.3 -clock PHI2 -add_delay {IN1}` gives IN1 delays relative to both PHI1 and PHI2, whereas `set_input_delay 2.3 -clock PHI2 {IN1}` would have removed the PHI1 delay. The same applies to edges of one clock: `set_input_delay 4.5 -clock PHI1 -max -rise -add_delay {IN1}` and `set_input_delay 4.0 -clock PHI1 -max -fall -add_delay {IN1}` accumulate rather than overwrite. This matters whenever a port genuinely receives data launched by different clocks, or on different edges, in different modes or paths - each launching condition needs its own budget.

One port, two launching clocks, two constraints



without `-add_delay` the second command replaces the first

In PD / signoff. You hit this on any interface fed from two source domains or characterized separately for rise and fall. The check is `report_port`: it should list every delay you intended on the port. If only the last-specified clock appears, the earlier ones were overwritten and their paths are now mis-budgeted or unconstrained.

Trap. Omitting `-add_delay` on the second and subsequent commands. The first clock's delay is removed with no error, so the PHI1 paths become unconstrained or mis-budgeted while the port still looks constrained - a silent under-constraint that produces no violations rather than wrong ones.

Expect next. When does a port legitimately need input delays from more than one clock?

Constraints / I/O Timing · `set_input_delay`, `set_output_delay`, `add_delay`, `report_port`

ADVANCED



42 Walk through how PrimeTime performs a full timing update internally, from reading data to computing slack.

It builds the timing graph after `link_design`, propagates the defined clocks to every register clock pin, calculates arc delays with slew propagation, propagates arrivals forward and required times backward while honoring exceptions, then subtracts them to get slack.

A full timing update runs as a sequence of conceptual stages. First the design graph: after `link_design` the linked netlist becomes a graph of cells and nets, from which PrimeTime builds a timing graph whose nodes are pins and whose edges are timing arcs - cell arcs from the library, net arcs from the parasitics. Second, clock propagation: the defined clocks from `create_clock` and the generated clocks are propagated through the clock network to every register clock pin, establishing each register's clock, its edges and its latency, source plus network; case analysis and clock gating prune that propagation. Third, delay calculation: each timing arc gets a delay - cell arcs from library tables interpolated on input slew and output load, where load is parasitic plus pin capacitance, net arcs from the parasitics or the wire-load model - with slew propagated so each stage's output slew feeds the next stage's input slew, and with crosstalk delta delays computed iteratively when SI is enabled. Fourth, forward propagation: arrival times accumulate from all startpoints, keeping both max, latest, and min, earliest. Fifth, backward propagation: required times propagate back from all endpoints, built on capture clock edges and setup or hold requirements, honoring exceptions - false paths pruned, multicycle paths shifting the capture edge. Sixth, slack at each pin: required minus arrival for setup, arrival minus required for hold. Finally reporting, where path-based reports trace the worst paths.

In PD / signoff. Knowing the order explains runtime and error behaviour that otherwise looks arbitrary: constraints that arrive after the clocks are propagated, or parasitics read after delay calculation, force stages to be redone, and an SI-enabled run costs more because delta delays iterate. It also tells you which stage a complaint comes from - a missing clock is a propagation-stage problem, not a reporting one.

Trap. Assuming the order of the stages is cosmetic and constraints can be applied in any sequence. Clock propagation has to precede arrival propagation, because launch edge time and latency anchor the arrival computation and capture edge and latency anchor the required time - without it the tool has nothing to compute from.

Expect next. Why must clocks be propagated before arrival times are computed?

STA Fundamentals / Slack & Arrival · `link_design`, `create_clock`

EXPERT

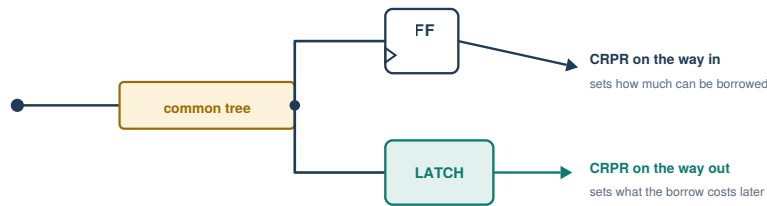
43 Explain the transparent-latch two-CRPR mechanism's effect on time borrowing, with the sign of each effect.

PrimeTime computes a separate Rise CRPR and Fall CRPR and shifts each latch edge later. Opening-edge pessimism shifts the opening later and reduces borrow capacity; closing-edge pessimism shifts the closing later and increases the maximum borrowing allowed.

A transparent latch endpoint has two edges that matter: an opening edge, after which data can start borrowing time, and a closing edge, by which borrowing must end. Because the two edges are different polarities on the clock, PrimeTime computes a Rise CRPR and a Fall CRPR separately and shifts each edge by its own pessimism. The two shifts then act in opposite directions on borrowing. Pessimism on the opening edge shifts the opening later, which reduces the amount of time available to borrow - and it also worsens the slack of any non-borrowing path referenced to that edge. Call that a tightening on borrow capacity. Pessimism on the closing edge shifts the closing later, which increases the maximum borrowing allowed and reduces the violation of a path that fails setup at the latch. That is a loosening. So one credit tightens the borrow window from the front while the other opens it at the back, and their effects do not cancel or combine into a single number. That is precisely why a latch needs both CRPRs computed separately rather than one lumped value: a single average would misstate both the borrow capacity and the setup slack at the latch, and it would get the sign of at least one of them wrong for any path that actually borrows.



A latch sits between two clock relationships, so it gets two credits



The two credits carry opposite signs on the borrow, so latch borrowing and CRPR have to be reasoned about together.

In PD / signoff. You need this when reading a latch-based path report where borrowed time and slack both look off. Knowing the opening and closing edges carry independent CRPR shifts, in opposite directions, tells you whether a reported borrow limit is a real timing constraint or an artifact of clock pessimism before you go asking for a latch-side ECO.

Trap. Assuming one lumped CRPR value applies to the whole latch endpoint. Because opening-edge pessimism reduces borrow capacity while closing-edge pessimism increases the borrowing allowed, a single average misstates both, and can predict the wrong sign for the change in either borrowed time or the setup slack at the latch.

Expect next. If the Rise and Fall CRPR values at a latch differ substantially, what does that tell you about the clock path feeding it?

Operating Conditions / General

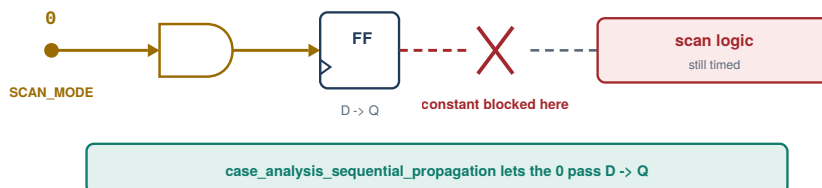
ADVANCED

44 Construct a scenario where forgetting sequential propagation makes a scan-disable silently ineffective.

Set `set_case_analysis 0 [get_ports SCAN_MODE]` without enabling sequential propagation on the scan flops and the constant stops at their TE pins: the CP->TI scan arcs stay enabled and the scan chain is still timed, with no warning.

Here is the scenario. You write `set_case_analysis 0 [get_ports SCAN_MODE]` intending to remove the scan chain from the analysis, but you never enable sequential case-analysis propagation on the scan flops - the global default is `never`, and you did not run `set_case_sequential_propagation` on them. The constant 0 does propagate forward through the combinational logic and does reach the scan flops' TE pins. What it cannot do is turn that TE=0 state into the conclusion that the scan input path is inactive, because that translation requires sequential case analysis to be active on those flops. Without it PrimeTime treats them as ordinary sequential cells, the CP->TI scan arcs remain enabled, and the scan chain is still timed. Your scan-disable did nothing, silently. The visible symptoms are indirect: extra scan-network paths that are meaningless in functional mode, possibly producing false violations, and definitely wasting runtime. You would only catch it two ways - by running `report_disable_timing` and noticing the CP->TI arcs do not appear as disabled, or by seeing scan paths show up in `report_timing`. The fix is to enable propagation on the right cells, for example `set_case_sequential_propagation [get_cells scan_ff_*]`, so that the TE constant actually disables the scan arcs, and then verify with `report_disable_timing`. This is the canonical trap that case analysis does not cross flops by default.

A constant stops at the first flop unless sequential propagation is on



Without it the scan-disable prunes only the cone in front of the first register.



In PD / signoff. Any functional-mode signoff run on a scan-inserted netlist hits this. After sourcing the mode constraints you check `report_disable_timing` for the expected CP->TI disables before you trust a single timing report; if they are absent you add targeted `set_case_sequential_propagation` on the scan flops. It is the difference between a clean functional QoR review and hours spent debating scan-path violations nobody needs to fix.

Trap. Believing a correctly applied port constant is sufficient proof the scan chain is off. The value is on the port, `report_case_analysis` looks right, and yet the arcs were never disabled because propagation stopped at the flops. The falsifiable check is whether CP->TI arcs appear in `report_disable_timing`.

Expect next. Why is flipping the global setting to `always` a dangerous way to fix this instead of using per-cell `set_case_sequential_propagation`?

Case & Mode Analysis / Case Analysis · `set_case_sequential_propagation`, `report_disable_timing`, `set_case_analysis`, `get_ports`

EXPERT

45 How should clock modeling rigor differ for a safety-critical versus a consumer design?

The clock mechanics are identical; what differs is exhaustiveness, margin justification and traceability. Safety-critical adds evidenced completeness verification, conservative documented margins, individually justified reviewed clock groups, an auditable clock spec and independent review.

Both designs use the same commands and the same modelling; the difference is how much you must prove. For safety-critical work, drive clock rigor to the maximum. Completeness is exhaustively verified as a mandatory, evidenced gate - zero unlocked registers, every generated clock defined and confirmed, every clock-group relationship justified and reviewed. Margins are conservative and justified: uncertainty, OCV and jitter margins chosen deliberately and documented, dynamic jitter modelled, worst-case early and late latency applied. Relationship discipline is rigorous, because clock groups are where silent gaps live: every asynchronous or exclusive group is individually justified, since an over-broad group hides real crossings and that is unacceptable here, and every synchronizer at an asynchronous crossing is verified to exist. The clock spec is traceable - which definitions and relationships were used for each qualified signoff, provably consistent with the fabricated clock tree, retained as certification evidence. The clocking gets independent review, and signoff is propagated against the real tree with no ideal shortcuts. The framing is that the clocks are part of the safety case: a clock error is a potential undetected hazard, and skew-driven hold errors especially so. A consumer design uses the same methodology with proportionate rigor - complete, propagated, correctly related clocks with verification - but lighter formal justification, traceability and independent review, optimizing cost and schedule, because there a clock error is a quality and rework risk rather than a hazard.

In PD / signoff. This decides what your signoff package contains, not how you run PrimeTime. On a safety-critical block you archive the exact clock constraints per qualified run, a written justification for every `set_clock_groups` declaration, `check_timing` evidence of zero unlocked registers, and proof the propagated latency came from the fabricated tree. On a consumer block you run the same checks but do not build the evidence trail.

Trap. Assuming safety-critical rigor means bigger margins. The distinctive requirement is provable completeness and justified relationships - and the specific hole is an unjustified or over-broad asynchronous or exclusive clock group, which `check_timing` will never flag because the exclusion looks deliberate.

Expect next. Why is clock-group discipline especially critical in a safety-critical design?

Clocks / Generated Clocks · `set_clock_groups`, `check_timing`

ADVANCED

46 Why might `report_mode` show a mode ENABLED with Reason default even though you set case analysis intending to disable it?

`Reason default` means no mode-selection method actively chose or excluded that mode, so it fell to the group's default state - the tell that your intended `set_case_analysis` never actually reached the mode's controlling condition.

`Reason default` is a statement about the selection mechanism, not the mode: it means nothing actively chose or excluded this mode, so it fell back to the group's default or all-enabled state. If you set case analysis intending to disable it and still see `default`, there are four likely causes. First, the case value never reached the mode's controlling pin - it needed to propagate through a sequential cell, the default for propagation is `never`, and you did not enable it, so the mode-select condition was never evaluated as false. Second, you set the constant on the wrong pin through a typo or a hierarchy mistake, so the mode's `when` or `sdf_cond` condition never saw it. Third, the mode belongs to a different group than you assumed, so selecting a mode elsewhere had no effect on it because the groups are independent. Fourth, the value was overridden by a direct `set_mode` or a higher-precedence value.



Whichever it is, the `default` reason is the tell that your intended selection never took effect. Cross-check with `report_case_analysis` to see whether the value landed on the right pin, `report_disable_timing` to see what actually got disabled, and the mode's Liberty condition to see what it was looking for. It is the mode-analysis analogue of an ignored exception: you set something, it did not do what you thought, and the Reason column is how you catch it.

A mode can be ENABLED without you enabling it

mode	status	reason
FUNC	ENABLED	default
TEST	disabled	case analysis

nothing you wrote turned this on

Reason 'default' means the mode was never explicitly selected - the tool fell back to it, so your case analysis did not choose it.

Read the reason column, not just the status column.

In PD / signoff. During mode-setup review an engineer reads the Reason column of `report_mode`, not just the Status column, for every mode that matters. An unexpected `default` sends them to `report_case_analysis` to check where the constant landed, and the constraint file is fixed before the view is used, because otherwise the group runs all-enabled and the analysis is pessimistic or ambiguous.

Trap. Reading only the Status column and concluding the mode setup worked. A mode can be enabled with reason `default` purely because your selection never took effect, which is a silently broken setup rather than a deliberate choice, and Status alone cannot distinguish the two.

Expect next. Given a mode showing `default`, how would you distinguish a propagation failure from a wrong-pin typo using the available reports?

Case & Mode Analysis / Case Analysis · `report_mode`, `set_case_analysis`, `set_mode`, `report_case_analysis`

EXPERT

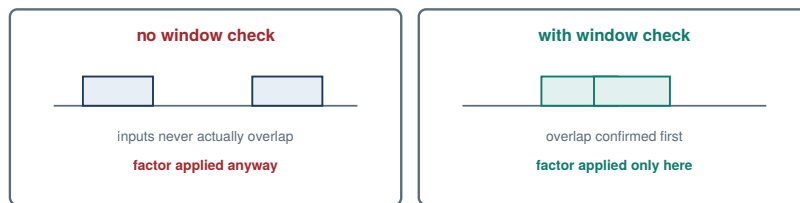
47 Prove that MIS, done wrong (full factor with no window check), can be either safe - but-costly or actually wrong - and state which.

Applying the full MIS factor with no arrival-window check is the optimistic extreme for hold: MIS shortens min delay, so an unconditional speed-up can make data arrive earlier than reality and mask a genuine hold violation.

Start from what MIS actually moves. Multiple-input switching applies a speed-up factor (less than 1) to a gate's delay when several inputs can switch together, and it is scoped to min delay - the hold quantity. The arrival-window filter gates that speed-up on whether the other inputs can really coincide with the input under analysis. Disable the filter and you apply the full speed-up factor unconditionally, which is the same as asserting that the other inputs always switch simultaneously. For hold, a larger min delay is the safe direction, and the window filter can reduce the speed-up where the inputs do not overlap, giving a larger and therefore safer min delay. Full factor with no window check gives the smallest min delay of all, so it is the optimistic and less safe extreme, and it can hide a real hold violation. That is worth stating plainly even when the option is described elsewhere as the more pessimistic or faster setting - that phrasing is about the setup-side or general delay perturbation and about runtime, not about the hold quantity MIS protects. The right reading is that disabling the overlap gating trades accuracy for runtime, and you must verify it does not optimistically clear real hold violations. The interview habit to voice: always tie the words pessimistic and optimistic to a specific check and the specific delay being moved. MIS moves min delay, so reason about hold.



The same coefficient, applied with and without a window check



wrong on setup: costly. wrong on hold: dangerous.

In PD / signoff. You meet this when tuning MIS settings for hold signoff runtime on a large post-route run. Before accepting the faster, no-window-check configuration, re-run the hold report with the arrival-window filter active and compare min-delay slacks on the tightest endpoints. If any endpoint moves from passing to failing, the faster setting was clearing violations optimistically and you keep the overlap gating.

Trap. Reading a claim that an option is more pessimistic and applying it to whichever check happens to be in front of you. The full-factor MIS setting is more optimistic for hold specifically, because MIS reduces min delay - so quoting the general pessimism and then signing off hold on it is exactly the trap.

Expect next. For a given gate, how would you demonstrate from the delay numbers alone that the arrival-window filter made the min delay larger rather than smaller?

Delay Calculation / Interconnect & Ceff

ADVANCED

48 How do you make PrimeTime remember the source file and line number of each exception, and what's the hard restriction?

Set `sdc_save_source_file_information` to true - it defaults to false - and PrimeTime records the file and line of every exception command. The hard restriction is that it can only be changed before any exception has been input.

PrimeTime can tag each timing exception with where it came from, but you have to ask for it up front. The variable is `sdc_save_source_file_information`, and its default is false, so nothing is recorded unless you turn it on. The restriction is one of ordering: the value can only be changed before any exception has been input. That means it belongs in your setup file, or on the very first lines of the session. Once even a single exception command has run, an attempt to set the variable produces a CMD-013 error and the value is left unchanged - there is no way to retrofit the information onto exceptions already in memory, so the only recovery is to restart the session with the variable set first. The scope of the tracking is the four exception commands: `set_false_path`, `set_multicycle_path`, `set_max_delay` and `set_min_delay`. Once enabled, you read the location back with `report_exceptions`, or with `report_timing -exceptions`, which prints an annotation of the form `[ location = multi.tcl:17 ]` next to the exception. The value of all this is traceability. On a design whose constraints arrive from several teams and several generated Tcl files, the difference between knowing that a path is masked and knowing which line masked it is the difference between a five-minute conversation and an afternoon of grepping.

In PD / signoff. On a large SoC you inherit an SDC split across dozens of files plus generated Tcl. Enabling this in the tool setup file means every `report_exceptions` line, and every dominant or overridden exception in `report_timing -exceptions`, names the file and line that created it - so when a path turns out to be masked you go straight to the owner and the commit instead of searching thousands of constraint lines.

Trap. Trying to enable it after sourcing the SDC. The command returns CMD-013, the variable stays at false, and engineers conclude the feature is broken rather than mis-ordered. Setting it mid-session also does not retroactively tag exceptions already read in - you must restart with the variable set before the first exception command.

Expect next. If an exception command sits inside a `foreach` loop, which line number does PrimeTime report?

Timing Exceptions / False Paths · `set_false_path`, `set_multicycle_path`, `set_max_delay`, `set_min_delay`

EXPERT

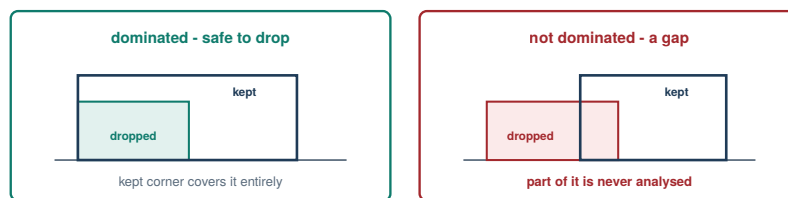


49 A team proposes reducing the signoff corner set to save runtime near a deadline. How do you respond?

Support reduction only where corner-analysis proves the dropped corners are dominated, push acceleration instead, and refuse blind dropping - an unanalyzed corner is a silent coverage gap where a real failure escapes to silicon with no warning.

Start by naming the risk. Each corner represents a real operating condition, and paths are worst-case at different ones - setup at a slow corner, hold at a fast corner, SI at a particular RC corner. Drop a corner that is worst-case for some paths and those paths are simply never verified at their worst condition, so a real violation there escapes with no signoff warning at all. That means the whole question turns on distinguishing redundant corners from load-bearing ones. Some corners are genuinely dominated, always better than a corner you are keeping, and can go safely; others are the worst case for some paths and must stay. Corner-analysis, which identifies dominance and worst-case coverage, is what tells them apart - not a deadline-driven guess. So the answer is to reduce intelligently: use corner-analysis to find the dominant set that covers all worst cases, keep those, and drop only the provably dominated ones, which cuts runtime without losing coverage. Better still, prefer acceleration over reduction - distribute the runs so scenarios execute in parallel, apply PBA selectively to violators, and use ML acceleration to cut wall-clock time with nothing dropped. And if a real coverage reduction is genuinely forced, it becomes an explicit, documented, escalated risk decision naming which corners were dropped and what is unverified, never a silent omission in a script.

Dropping a corner is only safe if another one dominates it



An unanalysed corner is not a smaller risk - it is an unknown one, and silicon finds it.

In PD / signoff. This lands as a concrete pre-tapeout decision: run corner-analysis to rank dominance, keep the covering set, and put the rest of the runtime problem on the compute farm through distributed multi-scenario analysis with PBA restricted to GBA violators. The output is a scenario list you can defend line by line at the signoff gate.

Trap. Dropping the corners that historically never showed violations. Historical cleanliness is not dominance - it can just mean those paths were not critical in the previous netlist - and because the reduced run reports nothing about what it did not analyze, the gap leaves no evidence anywhere.

Expect next. When is corner reduction provably safe versus dangerous?

STA Fundamentals / STA in the Flow

ADVANCED

50 Explain why `set_aocvm_table_group` / `read_ocvm` after a timing update triggers a full update, and the flow implication.

Derating factors are an input to delay calculation, so changing tables or table groups alters delays on a broad set of arcs and arrivals propagate transitively - there is nothing to patch incrementally, so a full update is forced. Set all variation data before the first update.

AOCV and POCV derating factors sit upstream of delay calculation: they multiply, or for POCV parameterize, the delay and sigma of every affected arc, and those arc values feed arrival propagation across the whole timing graph. When you change the table group with `set_aocvm_table_group`, load new tables with `read_ocvm`, or remove a group, the derate on a wide set of arcs changes at once. Because arrivals propagate transitively, the tool cannot reason about which handful of endpoints moved - the change ripples design-wide - so it invalidates and recomputes full timing rather than doing an incremental update. The flow implication is straightforward: set your variation data and grouping once, up front, before the first `update_timing` or `report_timing`, so you pay the full update a single time. Interleaving `read_ocvm` or `set_aocvm_table_group` with reporting commands forces repeated full updates, the same anti-pattern as `report_exceptions` or a global parasitic scaling triggering full updates. Batch all variation setup, then analyze. If you genuinely must iterate on table groups - tuning the derate on a reused block, for example - expect each change to



cost a full update and script accordingly, or do the iteration in a dedicated scenario setup phase. The general principle: anything that changes derating broadly is a full-update operation, so plan the flow so those land at deliberate checkpoints, not in the middle of a report loop.

What forces a full timing update instead of an incremental one



Read the variation data before the first update, not after it.

In PD / signoff. In a signoff script you place all `read_ocvm` and `set_aocvm_table_group` calls in the constraint and variation setup section, before the first `update_timing`. When runtime blows up on a large design, the first thing to check is whether a derate command sits below a reporting command in the script - that single ordering error can double or triple the run.

Trap. Loading or regrouping OCVM data between reports, on the assumption that a derate change is a cheap local edit. It is not - each such command discards the entire timing update, so a script that tunes derates inside a reporting loop pays a full update per iteration and looks mysteriously slow.

Expect next. Which other PrimeTime operations force a full update rather than an incremental one, and what do they have in common with a derate change?

Variation / AOCV Tables · `read_ocvm`, `set_aocvm_table_group`, `update_timing`, `report_timing`

INTERMEDIATE

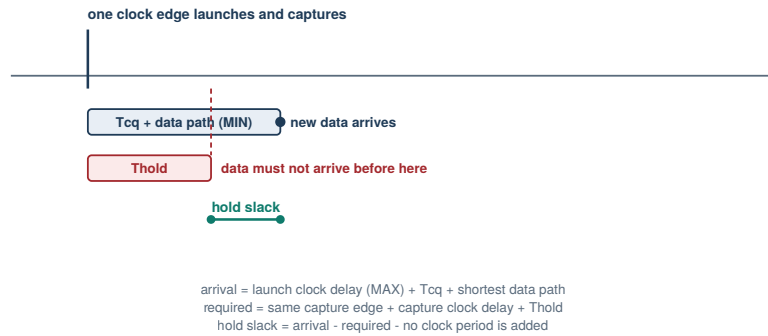
51 Explain the complete hold check and why a long clock path can cause a hold violation.

A hold check verifies data launched by a clock edge does not reach the capture flop too early and corrupt what that same edge should capture. It uses the shortest data path against the latest capture edge, with no clock period added.

A hold check on FF1 -> FF2 asks whether data launched from FF1 arrives too early - specifically, whether new data launched by a clock edge arrives and corrupts the value FF2 is supposed to capture from that same edge, which was launched the previous cycle. That same-edge framing is the key difference from setup: no clock period is added, so it is the short paths and the clock skew that determine hold, not the frequency. The arithmetic is $\text{data_arrival} = \text{clk_launch_delay} + \text{Tcq}(\text{FF1}) + \text{data_path_delay}(\text{min})$ and $\text{data_required} = \text{clk_capture_delay} + \text{Thold}(\text{FF2}) + \text{uncertainty}$, with $\text{hold_slack} = \text{data_arrival} - \text{data_required}$ and slack greater than or equal to zero meaning the check is met. Being a min-delay check, the tool uses the shortest data path so data is as early as possible, and considers the clock delay that makes the capture edge as late as possible, maximising hold risk. That is why a long clock path causes violations. A large capture clock delay pushes the capture edge later, so newly arriving data has more time to reach FF2 before that late edge - exactly the corruption condition. Work the numbers: with $\text{clk_launch}=0.2$, $\text{Tcq}=0.2$, $\text{data_path}(\text{min})=0.4$, arrival is $0.2 + 0.2 + 0.4 = 0.8$. With $\text{clk_capture}=1.0$ (long), $\text{Thold}=0.2$ and $\text{uncertainty}=0.05$, required is $1.0 + 0.2 + 0.05 = 1.25$. Hold slack is $0.8 - 1.25 = -0.45$, a violation caused purely by clock skew.



Hold check: earliest data arrival vs the same capture edge



In PD / signoff. Hold is closed after clock tree synthesis and again post-route, because it depends on real skew. You run `report_timing -delay_type min` and read whether the failure comes from the data path being short or from capture clock delay exceeding launch. That distinction decides the fix: buffer insertion in the data path, or rebalancing the clock tree so the capture edge stops arriving late.

Trap. Adding a clock period to the hold required time, or believing hold gets easier at a lower frequency. Hold is a same-edge check with no period term, so slowing the clock does nothing for it - and assuming a long capture clock is harmless inverts the risk, since it is exactly what lets new data race in.

Expect next. Does the hold check compare against the next clock edge or the same edge that launched the data?

STA Fundamentals / Hold Checks · `report_timing`, `delay_type`

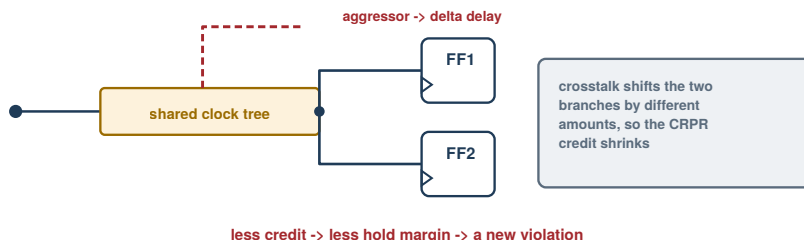
EXPERT

52 A path fails hold only after you enabled crosstalk analysis. Could CRPR be involved, and how would you investigate?

Possibly. If it is a zero-cycle hold check, crosstalk on the common clock segment should be CRPR-removed, and if that credit is missing you see false hold pessimism. The key discriminator is whether the check is zero-cycle.

CRPR could be involved, but only under one condition. If the check is zero-cycle - the same edge launches and captures, as with a divide-by-2 or certain adjacent-flop structures - then crosstalk-induced delay on the common clock segment should be removed by CRPR, because the aggressor perturbs launch and capture identically. If that credit is not being applied correctly, the extra hold pessimism you are seeing is false. Investigate in order. First confirm the check really is zero-cycle by reading the launch and capture edges out of `report_timing`; do not infer it from the topology alone. Second, run `report_crpr` and see whether the crosstalk delta on the common segment is actually being removed. Third, verify `timing_remove_clock_reconvergence_pessimism` is true, since the whole mechanism is off otherwise. Fourth - and this is the branch people skip - if the check turns out not to be zero-cycle, then CRPR is behaving correctly by not removing the crosstalk delta, because launch and capture are different edges and the aggressor's effect on each is not guaranteed to match. In that case the hold failure is likely real signal integrity, and you should pursue SI fixes such as shielding or increased spacing rather than hunting for a missing pessimism credit. The zero-cycle question is the discriminator that tells you which of those two entirely different debugs you are in.

A hold failure that only appears once crosstalk is on



Check whether the CRP value moved before hunting for a real data-path problem.



In PD / signoff. This is post-route hold debug once crosstalk analysis is switched on and the violation list grows. For each new failure you read the launch and capture edges, then either chase a missing CRPR credit or route the path to the SI team for shielding and spacing. Answering the zero-cycle question first is what keeps you from spending ECO budget on pessimism.

Trap. Assuming any crosstalk-induced hold failure on a shared clock segment must be CRPR pessimism. For non-zero-cycle checks CRPR correctly leaves the delta in, because different edges can genuinely see different aggressor activity, and removing it would be optimistic. The inverse error is shielding a zero-cycle path whose delta should have been credited back.

Expect next. For a non-zero-cycle check, what would you have to prove about the aggressor before you could argue the delta is a double-count?

Operating Conditions / General · report_timing, report_crpr, timing_remove_clock_reconvergence_pessimism

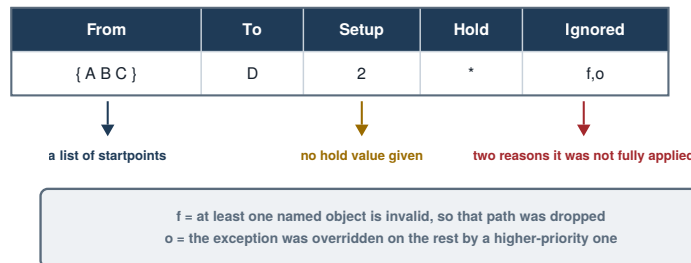
INTERMEDIATE

53 Read this exception report line and tell me what happened: From { A B C }, To D, Setup 2, Hold *, Ignored f,o.

A `set_multicycle_path 2 -from {A B C} -to D` was only partially applied: `f` means an invalid startpoint (C) was dropped, `o` means another path (A) was overridden by a higher-priority exception, and only B->D actually carries the multicycle value 2.

The report line reads `{ A B C } D 2 * f,o` - startpoints A, B and C to endpoint D, a setup value of 2, no hold value, and an Ignored column carrying two codes. That combination tells you a `set_multicycle_path 2 -from {A B C} -to D` was accepted syntactically but only partly took effect. The `f` code says at least one specified startpoint is invalid - here C, so the C->D path was dropped from the exception entirely. The `o` code says at least one path was overridden by a higher-priority exception - here A->D, which is already covered by a false path, and a false path outranks a multicycle path in the exception precedence order. What is left is B->D, and that is the only path actually enforced with the multicycle value 2. So of the three startpoints you wrote, one was never valid, one is excused by something stronger, and one behaves as intended. The skill being tested is decoding a partially ignored exception rather than assuming that because the command was accepted the whole thing took effect. Every field in that line is telling you something, and the Ignored column is the one that matters most.

Reading an exception report line



An exception can be accepted, listed, and still not doing what you meant.

In PD / signoff. This is exactly what you read after applying a batch of exceptions from a spec: check the Ignored column on every reported exception rather than just confirming the command returned. An `f` usually points at a stale or misspelled instance name after a netlist change; an `o` tells you a higher-priority exception already owns that path, so your new constraint is doing nothing there.

Trap. Assuming a command that produced no error was fully applied. A collection-style `-from {A B C}` can be two-thirds inert - one startpoint invalid, another overridden - and the only evidence is the Ignored column codes, which is why exception reports need reading rather than glancing at.

Expect next. Which exception types outrank a multicycle path, and what precedence order does PrimeTime apply?

Timing Exceptions / False Paths · set_multicycle_path

EXPERT



54 How would you decide which flow decisions to standardize organization-wide versus leave to team/project discretion?

Standardize what must be consistent, correct and comparable across the organization - flow structure, validation gates, reproducibility controls, sign-off criteria, result format, and core correctness settings. Delegate genuinely project-specific configuration, constraints, debugging and optimization.

The dividing line is whether something must be consistent and correct across the organization or is legitimately project-specific. Standardize the flow structure - the shared phase-script library and the 15-step order - so everyone runs the same validated process. Standardize the validation gates, `check_timing`, coverage and consistency checks, and make them mandatory everywhere so no team can omit the silent-gap checks. Standardize the reproducibility controls: configuration management, version pinning, and no dependence on personal files. Standardize the sign-off criteria, so that "signed off" means the same thing in every group. Standardize the result format and aggregation so results are comparable across teams. And standardize the core analysis-correctness settings, the ones that must be uniform for signoff to be valid, such as how variation and SI are modelled at a given node. Inconsistency or omission in any of these produces incorrect, non-reproducible or incomparable signoff, which is precisely why they cannot be left to taste. Then delegate, within that framework, the project-specific configuration - block list, the specifics of the mode and corner matrix, library versions for that project, input paths - along with design-dependent constraints and exceptions, each team's investigation and debugging approach, and project-specific optimizations inside the standard flow. Those legitimately differ per project and do not affect cross-organization consistency, so teams get flexibility inside a validated framework.

In PD / signoff. This shapes what a central methodology group ships versus what a block owner writes. The central release contains the phase scripts, the mandatory gates and the report format; the block owner supplies a config file and the block's SDC, and cannot silently disable a gate to make a review deadline.

Trap. Leaving the correctness settings or the validation gates to team discretion because they feel like local choices. Two blocks then sign off under different variation or SI modelling, their results are not comparable, and the chip-level conclusion drawn from combining them is unsound.

Expect next. Which flow elements should be left to project discretion, and what makes that delegation safe?

Tool Setup & Scripting / Scripting & `pt_shell` · `check_timing`

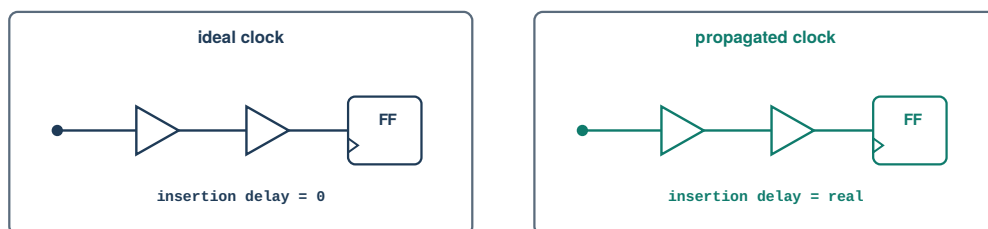
INTERMEDIATE

55 Why is on-chip variation derating only meaningful once clocks are propagated?

OCV works by derating clock latencies, deriving different launch and capture clock delays. With ideal clock latency there are no propagated clock delays to derate, so you approximate OCV's margin by inflating `set_clock_uncertainty` instead.

The reasoning is about what OCV has to act on. OCV's whole mechanism is applied to clock latencies: it derates the launch and capture clock paths differently so the two sides of a check no longer share an identical clock delay, and that asymmetry is what produces the pessimism you want. That requires propagated clock delays to exist in the first place. With ideal clock latency, the clock arrives at every register with no propagated delay to scale, so there is essentially nothing for OCV to derate and enabling it accomplishes very little. This is why OCV belongs with propagated clocks and is largely inert before clock tree synthesis. The consequence is not that you go without the margin, but that you model it differently. Before you have a real clock tree, you approximate what OCV would have contributed by inflating `set_clock_uncertainty` - a lumped budget standing in for launch-versus-capture clock variation you cannot yet derate path by path. It is a coarser instrument, since a single number replaces per-path derating, but it is the honest way to hold the margin while the clock tree does not exist. Once clocks are propagated, you switch to real OCV derating and pull the uncertainty budget back down accordingly, otherwise you are counting the same margin twice.

Ideal versus propagated clock



An ideal clock hides the tree so pre-CTS numbers stay comparable; a propagated clock uses the real buffer delays, which is the only mode that can expose skew and hold problems.



In PD / signoff. This determines how your constraints change across the flow. Pre-CTS, with ideal clocks, you hold clock variation in an inflated `set_clock_uncertainty`. Post-CTS, with propagated clocks, OCV derating takes over and the uncertainty value comes back down to real jitter and skew margin. Reviewing a post-CTS constraint file, a large pre-CTS uncertainty left in alongside active derating is double-counted margin.

Trap. Enabling OCV derating on an ideal-clock run and believing the margin is now covered. With no propagated clock delays there is nearly nothing to derate, so slack barely moves and the clock variation budget is effectively absent. The mirror error is keeping the inflated `set_clock_uncertainty` after clocks are propagated, double-counting the same variation.

Expect next. Once clocks are propagated and OCV derating is active, how would you decide how much of the inflated `set_clock_uncertainty` to remove?

Operating Conditions / On-Chip Variation · `set_clock_uncertainty`

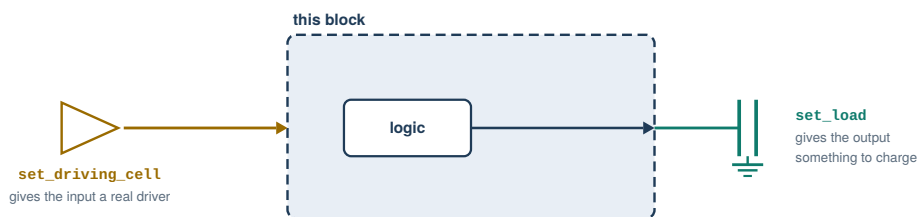
ADVANCED

56 How do you decide between `set_driving_cell`, `set_drive`, and `set_input_transition` for accurate boundary modeling?

Default to `set_driving_cell` when a library cell can represent the external driver, since it uses the cell's real load-dependent model. Fall back to `set_drive` for drivers not modeled as a library cell, and use `set_input_transition` when the slew is load-independent or for tool matching.

Choose based on what accurately represents the real external driver and on what modeling you actually have. `set_driving_cell` is the default and the most accurate option: it characterizes the port using an actual library cell's delay model, nonlinear NLDM or current-source CCS, so the port's transition and delay are computed as functions of the load and stay correct as the port capacitance changes. Since most ports are driven by standard cells that a library cell can represent, and since the input slew at the boundary propagates into all the downstream delays, this is the right choice for accuracy whenever a representative cell exists. `set_drive` is the fallback for a driver that cannot be characterized by a library cell - a custom or analog block, or IP not modeled as a Synopsys library cell - where a drive resistance is simply the best available representation, and you accept lower accuracy for nonlinear behavior. `set_input_transition` is appropriate when the transition is relatively independent of the design's load: a large external driver into a large external capacitance, where the design's comparatively small input load barely affects the slew, so a fixed transition is both accurate and simpler. It is also the right choice when you need to match another tool that only supports specifying input transition. Note that among these drive commands the most recent one wins - a later command removes the earlier one - so do not layer them and expect the accurate one to survive.

Modelling the electrical world outside the block



Without a driving cell an input transition is assumed ideal, and without a load an output sees almost no capacitance - both make the block look faster than it will ever be in the real chip.

In PD / signoff. At block constraint setup you check with `report_port -drive` which method is actually in effect on each input, after all your SDC files have sourced. Since the most recent drive command wins, a late generic `set_input_transition` on a wildcard port list can quietly wipe the careful `set_driving_cell` assertions - and the input slews feed every downstream delay, so it moves real numbers.

Trap. Applying `set_input_transition` on a port whose slew is actually load-dependent - a normal standard-cell driver into a normal load. The fixed slew ignores how the real driver responds to the port capacitance, so the boundary delay and every downstream delay are wrong in a direction nobody notices.

Expect next. In what specific case is `set_input_transition` actually the most appropriate choice rather than a compromise?

Constraints / Drive & Load · `set_input_transition`, `set_driving_cell`, `set_drive`, `report_port`

INTERMEDIATE

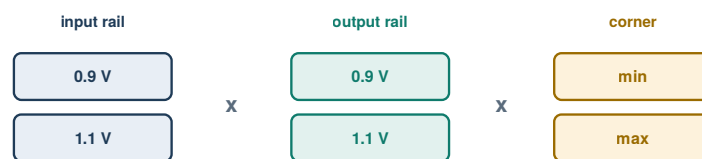


57 Why does a two-rail level shifter need at least eight scaling libraries?

Because a level shifter spans two power domains, its timing depends on two rail voltages plus temperature. Scaling on-grid across all three needs $2 \times 2 \times 2 = 8$ corner combinations, against $2 \times 2 = 4$ for a single-rail cell.

Scaling library count is set by the dimensionality of the cell's dependence, and a level shifter has one dimension more than an ordinary cell. A normal single-rail cell's timing depends on its one supply voltage and on temperature, so to scale on-grid you need the combinations of the two endpoints in each of those two dimensions: 2 for voltage times 2 for temperature, giving 4 libraries. A two-rail level shifter connects two different power domains, so its delay is a function of both rail voltages independently as well as temperature. That is a higher-dimensional scaling problem, and the on-grid corner set is the product across all three axes: 2 for rail 1, times 2 for rail 2, times 2 for temperature, which is 8 libraries. The reason you cannot economize here is that the two rails vary independently in a real multi-voltage design - one domain can be at nominal while the other is scaled down by DVFS - so a library set that only covers the two rails moving together leaves the mixed combinations uncharacterized, and those are exactly the operating points a level shifter exists to handle. Provide fewer than the eight and the tool has no on-grid data for some rail-voltage pairs it will legitimately be asked about.

Why a two-rail level shifter needs eight libraries



$2 \times 2 \times 2 = 8$ characterised libraries

Each supply the cell touches doubles the set, and the corner doubles it again.
Miss one and the tool silently falls back or refuses to scale.

In PD / signoff. During library setup for a multi-voltage design, count the level-shifter scaling libraries before you build the scenario matrix: eight per two-rail shifter cell, four for single-rail cells. A missing rail-voltage combination shows up later as a best-match failure or a fallback on the very cells that cross your DVFS domain boundaries, which is the worst place to lose accuracy.

Trap. Supplying four scaling libraries for a two-rail level shifter by treating the two rails as one voltage axis. That covers only the cases where both domains move together and leaves every mixed rail-voltage combination - exactly the DVFS operating points a level shifter is there for - without on-grid characterization data.

Expect next. Why can the two rail voltages not be collapsed into a single scaling dimension for a level shifter?

Delay Calculation / General

ADVANCED

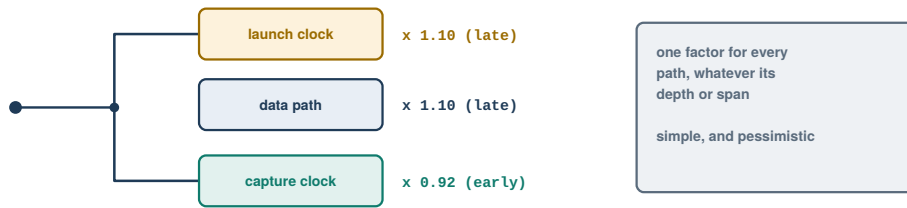
58 How does on-chip variation (OCV) fundamentally change the setup and hold analysis?

OCV derates parts of the path differently to build a worst case: for setup the data path and launch clock slow while the capture clock is early-derated; for hold the reverse. CRPR then credits back the pessimism double-counted on the shared clock path.

On-chip variation captures the reality that different parts of a chip run at slightly different speeds: even at one global corner, local process, voltage and temperature variation makes some cells and nets faster and others slower than nominal. Without OCV a check uses one delay per arc. With OCV, PrimeTime applies different derating to different parts of the path to construct the worst case. For a setup check it makes the data path slow, late-derated with larger delays, and the launch clock slow, while the capture clock is fast, early-derated - data as late as possible, capture edge as early as possible. For a hold check it does the opposite: data path fast, early-derated, and capture clock slow, late-derated - data as early as possible, capture edge as late as possible. That construction introduces pessimism, because the launch and capture clocks share common segments, the common clock path, and a single physical segment cannot realistically be both fast and slow at once. PrimeTime removes that over-pessimism with CRPR, clock reconvergence pessimism removal, crediting back the variation that was double-counted on the shared portion. More advanced modeling, POCV or parametric OCV, goes further by using statistical sigma-based derating per cell instead of flat factors, which reduces pessimism again - most visibly on deep paths, where a flat per-cell derate compounds unrealistically.



Flat on-chip variation derating



A 30-gate path is derated exactly as hard as a 3-gate path, even though its random variation partly cancels out.

In PD / signoff. At signoff this is a settings decision with real slack consequences: derate values applied per path segment, CRPR enabled so common-clock pessimism is credited back, and POCV where flat OCV is over-margining deep logic. When a path's slack improves substantially after enabling CRPR or POCV, the original number was pessimism, not a design problem, and no ECO was ever needed.

Trap. Running OCV derates without CRPR enabled. The shared clock segment is then derated slow for launch and fast for capture simultaneously, which is physically impossible, so nearly every register-to-register path carries invented pessimism - and teams spend area and power fixing violations that do not exist.

Expect next. What is CRPR crediting back, and why does it apply to almost every path?

STA Fundamentals / Setup Checks

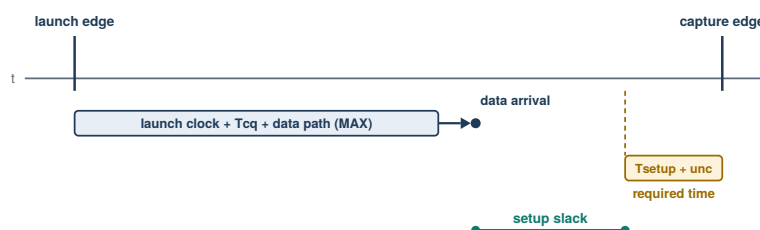
INTERMEDIATE

59 Why does the setup-check equation subtract the capture clock path but the hold-check equation also subtract it - aren't they opposite checks?

Both equations subtract clockpath2 because the capture-clock delay always shifts the capture edge later. For setup that later edge helps, for hold it hurts - same algebraic sign, opposite consequence for margin.

The sign in the equation and the effect on margin are two different things, and conflating them is what makes this question feel like a contradiction. Both checks subtract clockpath2 for the same physical reason: capture-clock delay shifts the capture edge later in time, which widens the launch-to-capture window. That is a single fact about the circuit, and it does not change depending on which check you are running. What changes is whether widening that window is good news. For setup, a later capture edge means the data has more time to arrive, so a larger clockpath2 relaxes the period requirement - the check gets easier. For hold, that same later capture edge is harmful, because it gives the new data more opportunity to arrive and be captured too soon, so a larger clockpath2 eats into hold margin - the check gets harder. Same algebraic sign in both equations, opposite consequence for the margin. This is exactly why OCV derates clockpath2 in opposite directions for the two checks. Since a larger clockpath2 helps setup, conservative setup analysis wants it scaled down; since a larger clockpath2 hurts hold, conservative hold analysis wants it scaled up. The derating direction is chosen by which way makes each check pessimistic, not by the sign the term carries in the equation. Read the equation for structure and the derate for intent.

Setup check: latest data arrival vs earliest required time



arrival = launch clock delay + Tcq + longest data path
 required = clock period + capture clock delay (MIN) - Tsetup - uncertainty
 setup slack = required - arrival (>= 0 means the path is met)



In PD / signoff. This is what lets you read a `report_timing` path and predict which way a clock-tree change will move slack. Adding capture-side clock latency helps the setup check on that endpoint and hurts its hold check, so a useful-skew fix for a setup violation shows up as new hold violations at the same register. It also explains why the same path report shows clock-path derate factors above one for one check and below one for the other.

Trap. Reasoning from the minus sign to the conclusion that clockpath2 must hurt setup and help hold, or expecting the two equations to carry opposite signs because they are opposite checks. The sign is identical; only the consequence flips, and the OCV derate direction follows the consequence rather than the sign.

Expect next. Given that, which direction does OCV derate clockpath2 for a hold check, and why is that the conservative choice?

Operating Conditions / On-Chip Variation · `report_timing`

EXPERT

60 How would you influence upstream teams (RTL, synthesis, library) to improve design-data quality for PrimeTime?

Push the library team toward accurate, complete, correctly-versioned libraries with the node's models, the synthesis team toward clean linkable netlists and a shared constraint source, and establish shared naming and versioning conventions plus a prompt feedback loop.

PrimeTime's data quality is determined upstream, so influence has to reach the teams producing the artifacts. With the library team the asks are well-characterised, correctly-versioned libraries with accurate delay tables and the right models for the node, meaning CCS and POCV data; consistent naming; fusion libraries where they simplify a multi-condition setup; clear version control and release notes so the intended libraries are the ones used; and validation that no cells or arcs are missing, since those become unresolved references or wrong timing. With the synthesis team the asks are clean, linkable netlists - no cells the libraries do not provide, and cell naming consistent with the libraries so `link_design` resolves every instance - plus a shared constraint source so signoff verifies the same constraints synthesis optimised to. With the RTL team it is clean design structure, documented clocking and modes, so the data and constraints are groundable, and STA-friendly design generally. Around all three, agree shared conventions for naming, versioning and handoff formats so data flows into PrimeTime without rework. And close the loop: report data-quality problems - unresolved references, library issues, netlist problems - back upstream promptly so they get fixed at the source rather than patched downstream every run. The argument that carries weight is the direct one: the libraries are the models the analysis computes with and the netlist is the design it assembles, so a defect in either produces wrong PrimeTime results, usually silently.

In PD / signoff. Concretely this looks like a handoff checklist enforced at each netlist and library drop - link cleanly with zero unresolved references, library version recorded, naming matching the mapping - and a standing defect channel back to the library and synthesis owners. It changes where debug effort goes: a naming mismatch becomes an upstream ticket rather than a local workaround repeated in every run script.

Trap. Absorbing upstream defects locally - hand-patching a `link_path` or a name mismatch run after run instead of reporting it. The defect stays in the source, every other team hits it too, and because unresolved references and wrong library versions produce no obvious error the same silent corruption keeps reaching other people's results.

Expect next. Why is a clean, linkable netlist a precondition rather than a nice-to-have from the synthesis team?

Design Data / Netlist & Libraries · `link_design`, `link_path`

INTERMEDIATE

61 What is the difference between recovery/removal checks and setup/hold checks?

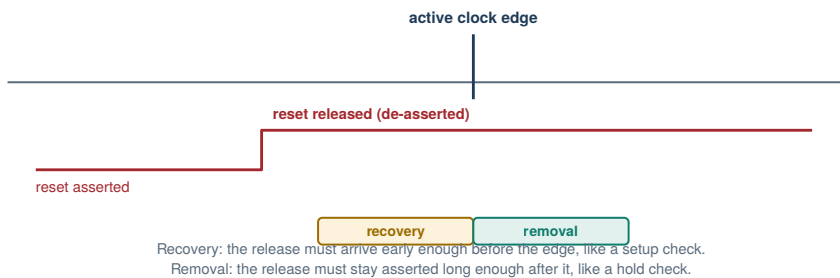
Recovery and removal are the asynchronous-control analogues of setup and hold, applied to a register's set or clear pin rather than its data pin: recovery is setup-like, requiring de-assertion early enough before the clock edge; removal is hold-like, requiring the signal to stay asserted long enough after it.

Recovery and removal do for asynchronous control what setup and hold do for data, and the difference is which pin they police. Setup and hold constrain the data pin; recovery and removal constrain the register's asynchronous set or clear - the preset or reset pin. A recovery check verifies that the asynchronous control signal, typically a reset, is de-asserted early enough before the next active clock edge for the register to resume normal clocked operation reliably on that edge. That makes it the setup analogue: the release must come with enough margin ahead of the edge. A removal check verifies the opposite side of the same edge - that the asynchronous control stays asserted long enough after the clock edge, or equivalently is not released too soon after it, so the edge does not catch the register mid-transition. That is the hold analogue: stable for long enough after the edge. So recovery is approximately setup and



removal is approximately hold, but for the timing of releasing the asynchronous control relative to the clock rather than for the data. What both are really protecting is the moment of handover: when an asynchronous reset is released, the release has to be properly timed against the clock, or the register does not come out of reset cleanly and can land in an indeterminate state.

Recovery and removal checks on an asynchronous reset



In PD / signoff. Reset release is where this bites in practice. Reset trees are large, often less well balanced than the clock tree, and recovery and removal failures show up as their own path groups in a full constraint report rather than under setup and hold. The usual fix is on the reset distribution - synchronizing or rebalancing the release - not on the datapath.

Trap. Swapping the two, or assuming an asynchronous reset needs no timing at all because it is asynchronous. The assertion may be asynchronous, but the release is checked against the clock edge, and an unchecked reset release is a classic silent path to registers coming out of reset in inconsistent states.

Expect next. Which of the two is the setup-like check, and on what kind of pin do both apply?

STA Fundamentals / Hold Checks

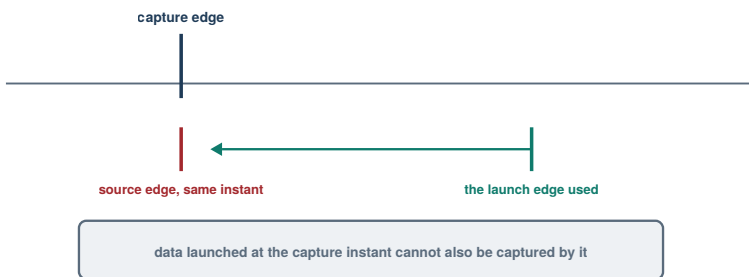
EXPERT

62 Explain, at the edge level, why a simultaneous source edge is not counted as the launch edge in the different-clock setup analysis, and why that's conservative-correct.

For a capture edge at time t , PrimeTime pairs the nearest source edge strictly before t , because data launched at t cannot also be captured at t . Reaching back to the previous edge yields the larger, more restrictive requirement.

PrimeTime builds the different-clock setup relationship by taking each capture edge at some time t and pairing it with the nearest source edge occurring strictly before t . A source edge at exactly t is excluded, and the reason is physical: in real hardware, data launched by an edge cannot also be validly captured by that same-instant edge, because there is no time for it to propagate through the launch flop and the logic between them. Treating the simultaneous edge as the launch would allot the path zero time and would describe nothing the hardware ever does. Excluding it forces PrimeTime to reach back to the previous source edge, and in the cases where this matters that yields a larger, more restrictive launch-to-capture requirement - the conservative and therefore safe choice for a setup check. The standard mixed-period case shows it directly: with CLK1 at period 10 and CLK2 at period 25, the capture at 30 pairs with the launch at 20, not with the simultaneous source edge at 30. The pairing that survives is the one that demands the most of the logic, and the simultaneous edge is skipped rather than treated as an easy pass. Understanding this is what lets you read a cross-domain report without suspecting the tool of arithmetic errors.

A source edge at the same instant is not the launch edge



In PD / signoff. This changes what you believe when you read the launch and capture times in a cross-domain `report_timing`. When the launch edge is not the one you expected on a mixed-period crossing, check whether a simultaneous source edge was skipped before you start doubting the clock definitions - the pairing PrimeTime chose is the conservative one and is usually right.

Trap. Deriving the setup relationship by hand and pairing the simultaneous edges on a mixed-period crossing, which gives a zero-length launch-to-capture window and a slack that disagrees with the tool. The follow-on error is assuming PrimeTime picked the wrong edge and adding a multicycle exception to 'correct' it.

Expect next. How does the same nearest-edge-before rule determine the hold relationship on that crossing?

Timing Exceptions / General · `report_timing`

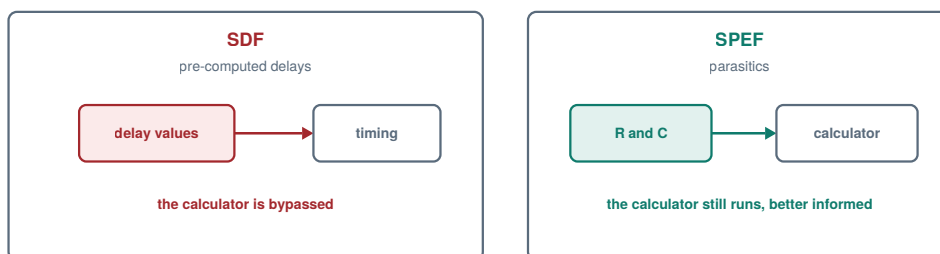
INTERMEDIATE

63 What is the difference between reading an SDF file and reading a SPEF file?

SDF carries pre-computed cell and net delays that PrimeTime reads with `read_sdf` and applies directly; SPEF carries extracted parasitic RC that PrimeTime reads with `read_parasitics` and computes delays from itself.

Both files back-annotate the design, but they hand the tool different things. An SDF file - Standard Delay Format - contains delay values that have already been calculated: the actual cell and net delays, and often the timing checks too. PrimeTime reads them via `read_sdf` and applies them as given; it does not recompute anything from parasitics. A SPEF file - Standard Parasitic Exchange Format - contains detailed parasitic RC data instead, the resistance and capacitance of each net as extraction produced it, read in with `read_parasitics`. From that, PrimeTime computes the net delays itself, and the load-dependent cell delays with them, using its own delay calculator. So the one-line difference is that SDF provides delays while SPEF provides the parasitics that delays are derived from. For signoff, SPEF is generally the preferred input, and the reason is control and self-consistency. When PrimeTime does the calculation, it does it with the actual libraries and the analysis settings in force for that run, including signal-integrity and crosstalk effects, applied uniformly to the extracted parasitics. An SDF's delays came from whatever tool wrote them, under whatever settings that tool had, and cannot pick up PrimeTime's crosstalk analysis afterwards - so they may simply not reflect the signoff conditions you are signing off under.

SDF replaces calculation; SPEF feeds it



That difference decides what your derates, thresholds and driver models can still affect after annotation.

In PD / signoff. This decides what you ask the physical-implementation side for. Post-route signoff runs read per-corner SPEF with `read_parasitics` so PrimeTime's own delay calculation and SI analysis apply consistently. SDF via `read_sdf` shows up mainly for correlation against another tool or for gate-level simulation timing, and any slack from an SDF run should be treated as reflecting the writer's settings, not yours.

Trap. Signing off on an SDF-annotated run and assuming it includes PrimeTime's crosstalk and analysis settings. It cannot - the delays were fixed before PrimeTime saw them, so SI delta delays and the current operating and variation settings have no way to influence them.

Expect next. Which of the two gives PrimeTime more control over the delay calculation, and why does that matter for signoff?

STA Fundamentals / STA in the Flow · `read_sdf`, `read_parasitics`

EXPERT



64 Explain why the SDF PORT construct and multidrive alignment are both about clock networks, yet solve different problems.

Both fight the pathology of a huge clock mesh, but the PORT construct compresses file size by collapsing near-identical INTERCONNECTs, while multidrive alignment forces one worst-case value on parallel driver cell arcs so the simulator does not fail.

They sit at different layers of the same problem: a large clock mesh or spine produces an unwieldy or unsimulatable SDF. The PORT construct, enabled with `sdf_enable_port_construct`, attacks file size and net count. A mesh has thousands of nearly identical INTERCONNECT delays, and the SDF 3.0 PORT statement collapses many INTERCONNECTs into one PORT when their delays differ by less than a threshold, 1 ps by default - the classic illustration being 16 INTERCONNECTs collapsing to 4 PORTs. That is representation compression that keeps the near-identical values, making the SDF smaller and faster to read and write. Multidrive alignment, enabled with `sdf_align_multi_drive_cell_arcs`, attacks simulation correctness instead. A net driven by multiple parallel drivers with slightly different cell-arc delays can cause a simulator failure through ambiguous multidriver timing; alignment unifies those driver arcs to a single worst-case value so the simulator sees one consistent delay. So PORT compresses the net and interconnect representation nearly losslessly within the threshold, while alignment forces a pessimistic common value onto driver cell arcs purely to keep the simulator happy. Different objects - interconnect versus cell arcs - different goals - size versus simulation safety - and critically, alignment is pessimistic and must never be read back into PrimeTime, whereas PORT is a benign compaction. They both cluster on clock networks because that is where massive parallelism creates both problems at once.

In PD / signoff. When writing SDF for gate-level simulation of a block with a clock mesh, an engineer enables `sdf_enable_port_construct` to keep the file loadable and `sdf_align_multi_drive_cell_arcs` when the simulator rejects the multidriven mesh nets. The consequence for the flow is that this SDF becomes simulation-only and is excluded from any PrimeTime read-back path.

Trap. Reading an alignment-processed SDF back into PrimeTime. Multidrive alignment deliberately replaces distinct parallel driver arc delays with one worst-case value, so re-annotating it injects pessimism into clock timing that no report will identify as artificial.

Expect next. What would you check before raising the PORT construct's 1 ps collapse threshold on a clock mesh?

Back Annotation / SDF Annotation

INTERMEDIATE

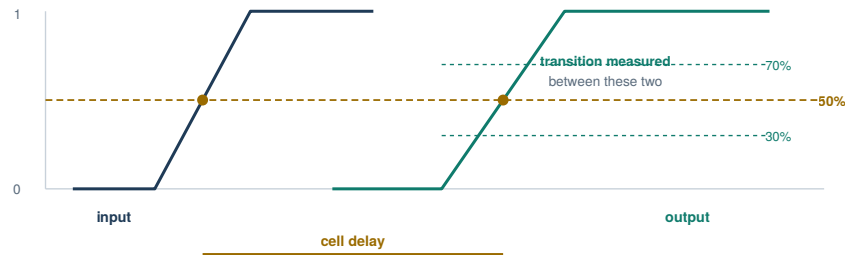
65 Delays and slews all look slightly off across the whole design. Why might trip points be the culprit?

Trip points are the measurement thresholds for every delay and slew, so an unexpected threshold shifts every number in the design consistently. Confirm with `report_delay_calculation -thresholds` on a suspect arc and `report_lib` to see which thresholds are actually in effect.

Trip points are where delay and slew are measured - the voltage thresholds that define the start and end of every arc and every transition. That makes them a global multiplier on your results rather than a local effect. If the threshold in force is not the one you assumed, then every delay and every slew in the design moves, and it moves consistently, which is exactly the symptom of everything looking slightly off rather than one path looking wrong. Two common causes fit that pattern. A library pin threshold may be overriding the value you thought applied, since thresholds come from a precedence order and the winning source is not always the one you set. Or a threshold value may be invalid, which forces the fallback to 5/95 - and if you were expecting something else, all your slews shift. Diagnose it directly rather than by inference. Run `report_delay_calculation -thresholds` on a suspect arc: that shows you the thresholds actually used for that specific delay computation. Then run `report_lib` on the library to see which thresholds the library defines and where in the precedence order they come from. Between the two you can establish both what is in effect and why, which is the information you need before you start suspecting the parasitics or the library version. The tell for this class of problem is uniformity: a threshold problem moves everything, so if only a handful of paths look wrong, look elsewhere.



Where delay and transition are measured



Change the threshold percentages and every delay and slew number changes with them - which is why they must match the library.

In PD / signoff. You reach for this when correlating PrimeTime against a golden signoff run or a SPICE spot-check and everything is off by a small consistent percentage. Run `report_delay_calculation -thresholds` on one arc in each tool setup and diff the thresholds before you diff anything else - it is a five-minute check that either explains the whole delta or rules out a whole class of causes.

Trap. Chasing a uniform delay or slew offset in the parasitics, library version or derating before checking thresholds. Trip points move every number in the design together, so a consistent global shift points at the measurement thresholds - and a library pin threshold overriding your assumption produces exactly that signature with no warning.

Expect next. What threshold does PrimeTime fall back to when a specified trip-point value is invalid?

Delay Calculation / Slew & Transition · `report_delay_calculation`, `report_lib`

